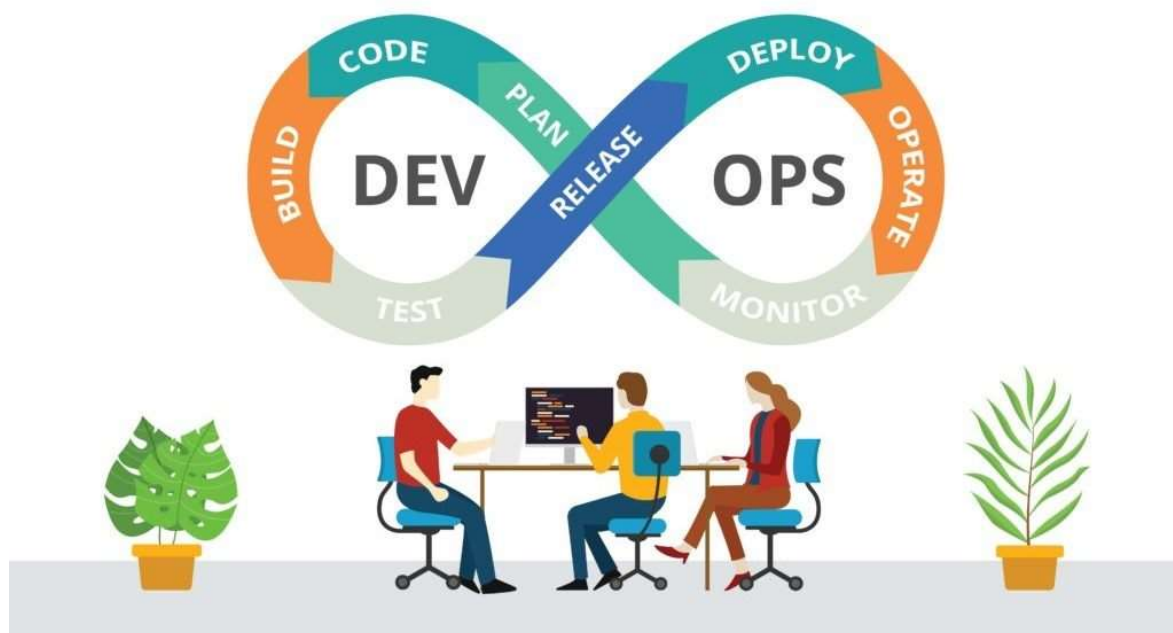


Методические рекомендации по проведению итоговой аттестации DevOps-инженер с нуля



Оглавление

Техническое обеспечение	3
Условия проведения	3
Критерии оценки.....	3
Список заданий	4
Описание решения заданий	6
Модуль 6. Администрирование баз данных	7
Модуль 7. Масштабируемость и отказоустойчивость.....	13
Модуль 8. Мониторинг, логирование и оповещение событий.....	22
Модуль 9. Виртуализация в DevOps	30
Модуль 11. Работа с Terraform: как управлять облачной инфраструктурой	35
Модуль 12. Конфигурационное управление. Что такое IaC.....	43
Модуль 13. Системы контроля версий. Распределённая система управления версиями Git	46
Модуль 14. Жизненный цикл ПО.....	49
Модуль 15. Практические навыки работы с Docker	54
Модуль 16. Микросервисы и микросервисная архитектура	56
Модуль 17. Оркестровка контейнеров, кластеры Kubernetes.....	58
Модуль 18. Kubernetes конфигурация развертывания.....	68
Модуль 19. Планирование безопасности для кластера Kubernetes.....	75

Техническое обеспечение

Для выполнения заданий итоговой аттестации необходимо:

1. Наличие персонального компьютера с любой операционной системой.
Предпочтительная операционная система Linux. При использовании операционной системы Windows, требования к железу: CPU не менее i7, 16GB RAM.
2. Персональный компьютер должен быть подключен к локальной сети, из которой доступна Технологическая платформа ПАО Ростелеком Базис. Digital Energy.
3. В технологической платформе должна быть заведена учетная запись для сдающего итоговую аттестацию с выделенными ресурсами CPU, RAM, HDD и достаточным набором образов операционных систем и специализированного программного обеспечения.
4. Наличие на персональном компьютере установленных программного обеспечения для работы с текстовыми файлами.
5. Наличие на персональном компьютере программного обеспечения для подключения к удаленному хосту по протоколу ssh (например: tilix, xshell, putty, superputty).
6. Доступ в Интернет.

Условия проведения

Итоговая аттестация проводится в очном формате с присутствием наблюдателя.

Сдающий аттестацию выполняет все задания самостоятельно с использованием конспектов и справочной информации, размещенной в сети Интернет. На выполнение работы отводится 5 (пять) астрономических часов.

Критерии оценки

Задания разбиты в соответствии с темами модулей курса. Каждое задание имеет уровень сложности в виде балла. Задание считается решенным, если полностью выполнено и достигнут поставленный результат. Итоговая оценка складывается из сумм баллов каждого решенного задания.

Список заданий

№ п/п	Модуль	Название модуля	Задание для проверочной итоговой аттестации	Уровень сложности (балл)	Время на выполнение (мин.)
1.	6	Администрирование баз данных	Развёртывание PostgreSQL версии не ниже 12. Создание базы данных из четырех столбцов произвольного названия. Наполнить произвольными тестовыми данными, не менее 10-ти значений.	3	10
2.	7	Масштабируемость и отказоустойчивость	На основе созданной БД из Модуля 6 создаем кластер из двух pod в реплике конфигурации Active/Passive	4	10
3.	8	Мониторинг, логирование и оповещение событий	Установка и развертывание Prometheus+Grafana+Node_exporter Настройка метрик нагрузки вашей ОС с отображением в Grafana	5	15
4.	9	Виртуализация в DevOps	Развертывание VM в Базисе на базе шаблона Ubuntu20.04 LTS Установка на нее пакетов: vim, mc, net-tools	3	10
5.	11	Работа с Terraform: как управлять облачной инфраструктурой	Развёртывание VM в БАЗИС на базе шаблона Ubuntu20.04 LTS с использованием Terraform* + provider для БАЗИСА	8	20
6.	12	Конфигурационное управление. Что такое IaC	Создание 2 VM на базе шаблона CentOS 7.9, установка на них ansible и настройка. Создание ansible-playbook по созданию 2 папок, 1 пользователя, установки пакетов vim, mc с 1 VM на 2 VM	8	20
7.	13	Системы контроля версий. Распределённая система управления версиями Git	Установка git на VM, создать репозиторий в предварительно подготовленном сервере Git с пустым файлом Readme.md, клонировать репозитория на локаль, сделать commit с Readme.md в новой ветке с последующим merge branch в удаленном репозиторий в master.	5	15

№ п/п	Модуль	Название модуля	Задание для проверочной итоговой аттестации	Уровень сложности (балл)	Время на выполнение (мин.)
8.	14	Жизненный цикл ПО	Создание ВМ, установка на нём Jenkins, развёртывание с помощью Jenkins https://github.com/AliyunContainerService/redis-cluster на ВМ	8	20
9.	15	Практические навыки работы с Docker	Создание dockerfile на базе образа centos 7.9, в него установить PostgreSQL версии не ниже 12, добавить пользователя в БД, создать в БД 2 таблицы и наполнить их на 10 значений. Скрипты и наполнение можно взять из модуля 6. Собрать полученный docker image через docker build и запустить. Подключиться с хостовой машины к инсталлированному PostgreSQL утилитой psql.	12	45
10.	16	Микросервисы и микросервисная архитектур	Создать свой docker-compose file на базе v3, в котором описать развёртывание в одном контейнере PHP+ tomcat 8 и во втором пустой БД MariaDB, в которой будет замаплен локальный volume <code>./mysql/</code> . Собрать и запустить, показать страницу php из браузера и доступность порта MariaDB	8	20
11.	17	Оркестровка контейнеров, кластеры Kubernetes	Развернуть k8s v 1.26 кластер, состоящий из 1 master и 1 worker node и установить на worker node RabbitMQ через Helm-chart	8	20
12.	18	Kubernetes конфигурация развертывания	Написать Helm-chart по управлению k8s cluster из п.17, Установить в k8s cluster из п.17, Prometheus+Grafana	12	45
13.	19	Планирование безопасности для кластера Kubernetes	В k8s cluster из п.17 создать namespace vault и установить с помощью helm-chart hashicorp vault последней версии, настроить под него persistent volume с 10 GB. Создать ns app_test с лимитом 20mlcore, 2GB memory и установить туда haproxy с авторизацией в stats, ключ авторизации должен храниться в vault и быть вынесен в ENV pod сервиса	16	50

Описание решения заданий

Приведенное решение может служить для оценки полноты и правильности достигнутых результатов о время выполнения задачи сдающим.

Модуль 6. Администрирование баз данных

Задание: Развёртывание PostgreSQL версии не ниже 12.

Создание базы данных из четырех столбцов произвольного названия.

Наполнить произвольными тестовыми данными, не менее 10-ти значений.

Уровень: 2

Продолжительность: 15 мин.

Установка PostgreSQL

Официальная документация <https://www.postgresql.org/download/linux/ubuntu/>

Загрузки для Linux (Ubuntu)

PostgreSQL доступен во всех версиях Ubuntu по умолчанию. Однако Ubuntu «создает моментальные снимки» определенной версии PostgreSQL, которая затем поддерживается на протяжении всего срока службы этой версии Ubuntu. Другие версии PostgreSQL доступны в репозитории apt PostgreSQL.

Apt-репозиторий PostgreSQL

Если версия, включенная в вашу версию Ubuntu, не та, которую вы хотите, вы можете использовать [репозиторий PostgreSQL Apt](#). Этот репозиторий будет интегрирован с вашими обычными системами и системой управления исправлениями, а также будет предоставлять автоматические обновления для всех поддерживаемых версий PostgreSQL в течение всего срока поддержки [PostgreSQL](#).

Репозиторий PostgreSQL Apt поддерживает текущие версии Ubuntu:

- кинетический (22.10, не LTS)
- варенье (22.04, LTC)
- фокальный (20.04, LTC)
- бионический (18.04, LTS)

на следующих архитектурах:

- amd64
- arm64 (18.04 и новее; только версии LTS)
- i386 (18.04 и старше)
- ppc64el (только версии LTS)

Чтобы использовать репозиторий apt, выполните следующие действия:

```
# Create the file repository configuration:
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt
$(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'

# Import the repository signing key:
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc |
sudo apt-key add -

# Install the latest version of PostgreSQL.
# If you want a specific version, use 'postgresql-12' or similar
instead of 'postgresql':
sudo apt-get -y install postgresql
```

Дополнительные сведения о репозитории apt, включая ответы на часто задаваемые вопросы, см. на странице [PostgreSQL Apt Repository в вики](#).

Входит в дистрибутив

Ubuntu включает PostgreSQL по умолчанию. Чтобы установить PostgreSQL в Ubuntu, используйте команду `apt-get` (или другую команду apt):

```
apt-get install postgresql-12
```

Репозиторий содержит множество различных пакетов, включая сторонние дополнения. Наиболее распространенные и важные пакеты (при необходимости замените номер версии):

postgresql-client-12	клиентские библиотеки и клиентские двоичные файлы
postgresql-12	основной сервер базы данных
libpq-dev	библиотеки и заголовочные файлы для разработки интерфейса на языке C
postgresql-server-dev-12	библиотеки и заголовочные файлы для разработки бэкенда на языке C

Убедитесь, что служба запущена:

```
$ sudo systemctl start postgresql.service
```

Проверка статуса сервиса:

```
$ sudo systemctl status postgresql.service
```

Работа с аккаунтом PostgreSQL

PostgreSQL применяет термин «Роль». Практически это тот же аккаунт в Ubuntu. При запуске СУБД роли сервиса привязываются к одноименным аккаунтам в Unix-системах. Другими словами, при наличии роли в PostgreSQL, войти в СУБД можно с аккаунтом Ubuntu. При запуске СУБД генерируется аккаунт postgres, привязываемый к роли PostgreSQL.

Вариант 1

Войдем в аккаунт:

```
$ sudo -i -u postgres
```

После ввода команды видим подтверждение о переходе в аккаунт:

```
postgres@postgresdoc:~$
```

Откроем консоль Postgres:

```
$ psql
```

Консоль открыта, что подтверждается записью в начале строки:

```
postgres=#
```

Работа в СУБД ведется из консоли.

Узнать статус подключения:

```
postgres=# \conninfo
```

Возврат в аккаунт:

```
postgres=# \q
```

Вариант 2

Войдем в аккаунт postgres с sudo. Если сейчас находимся в аккаунте postgres, нужно выйти, набрав **exit**. В этом варианте перейдем в аккаунт **postgres с sudo**:

```
$ sudo -u postgres psql
```

Возврат в аккаунт:

```
postgres=# \q
```

Создание роли

Аккаунт postgres обладает правами администратора. Напишем **createuser**, эта команда сообщает, что мы добавляем новую роль. Чтобы указать имя роли и выдать суперюзера, применим флаг **—interactive**.

Запись будет такой:

```
postgres@postgresdoc:~$ createuser --interactive
```

Вариант работы без переходов между аккаунтами:

```
$ sudo -u postgres createuser --interactive
```

Вводим имя, выдаем суперюзера:

```
Enter name of role to add: tester
Shall the new role be a superuser? (y/n) y
```

Посмотреть другие ключи настроек:

```
postgres@postgresdoc:~$ man createuser
```

Роль создана, поднимаем БД.

Создание базы данных

Любому созданному аккаунту привязывается база данных с идентичным именем, то есть наш созданный tester начнет подключаться к базе данных tester.

Командой **createdb** добавим БД (поднимем новую базу PostgreSQL на Ubuntu), назвав ее tester:

```
postgres@postgresdoc:~$ createdb tester
```

Вариант работы без переходов между аккаунтами:

```
$ sudo -u postgres createdb tester
```

Переход в командную строку PostgreSQL с новой ролью

Работа в консоли PostgreSQL подразумевает наличие аккаунта Ubuntu с именем БД в PostgreSQL.

Добавим аккаунт Ubuntu, используя **adduser** (предварительно выйдя из аккаунта postgres), назвав аналогично новой роли:

```
$ sudo adduser tester
```

Добавив аккаунт `tester`, переключаемся на него и подключаемся к консоли:

```
$ sudo -i -u tester
$ psql
```

второй вариант:

```
$ sudo -u tester psql
```

Переключиться на другую БД:

```
$ psql -d postgres
```

Проверка статуса:

```
tester=# \conninfo
```

Увидим:

```
You are connected to database " tester " as user " tester " via socket
in "/var/run/postgresql" at port "5432".
```

Проверку желательно выполнять для разных пользователей с разными БД.

Создание таблицы с данными

Команда создания имеет вид:

```
CREATE TABLE table_name (
    column_name1 col_type (field_length) column_constraints,
    column_name2 col_type (field_length),
    column_name3 col_type (field_length)
);
```

Создавая таблицу, указываем ее имя, столбцы, их типы, ограничения размеров полей. В качестве непосредственного примера добавим таблицу комплекта сноубордиста:

```
tester=# CREATE TABLE snowboarder (
    equip_id serial PRIMARY KEY,
    title varchar (50) NOT NULL,
    company varchar (25) NOT NULL,
    size varchar (25) check (size in ('XS', 'S', 'M', 'L', 'XL',
'XXL'))
);
```

СУБД выводит информацию:

```
CREATE TABLE
```

- **Equip_id** — столбец с идентификатором типа serial и автоинкрементом. Ему присвоили свойство primary key, указывающее на использование не нулевых показателей.
- **Title** и **company** указывают наименование и фирму-производителя единицы комплекта.
- **Size** хранит размеры наименований комплекта, предлагая выбор одного из предложенных размеров.

Посмотреть таблицу:

```
tester=# \d
```

Вывод:

```
List of relations
Schema |          Name          |   Type   | Owner
-----+-----+-----+-----
public | snowboarder            | table    | tester
public | snowboarder_equip_id_seq | sequence |
tester (2 rows)
```

Создана таблица с переменной snowboarder_equip_id_seq, тип данных sequence. Переменная указывает на номера последовательности и генерируется автоматически. Вывести таблицу без переменной:

```
tester=# \dt
```

Вывод:

```
List of relations
Schema |      Name      | Type | Owner
-----+-----+-----+-----
public | snowboarder    | table | tester
(1 rows)
```

Работа с данными таблицы

Таблица есть, остается ее наполнить.

Добавление данных в таблицу

Состав комплекта нашего сноубордиста: snowboard (сноуборд), binding (крепления), boots (ботинки). Вызовем таблицу, где укажем столбцы и их значения. Добавим 3 строки:

```
tester=# INSERT INTO snowboarder (title, company, size) VALUES
('snowboard', 'burton', 'XL');
tester=# INSERT INTO snowboarder (title, company, size) VALUES
('binding', 'burton', 'XL');
tester=# INSERT INTO snowboarder (title, company, size) VALUES
('boots', 'burton', 'XL');
```

Пишем внимательно, избегая ошибок. Неверно использовать кавычки в названиях столбцов. Кавычки указываются в значениях столбцов. Значение столбца equip_id генерируется самостоятельно во время создания строки.

После добавления каждой строки СУБД подтверждает операцию:

```
INSERT 0 1
```

Вывод данных

Выведем таблицу с новыми данными:

```
tester=# SELECT * FROM snowboarder;
```

Увидим в ответ:

```
equip_id | title      | company | size
-----+-----+-----+-----
      1  | snowboard | burton  | XL
      2  | binding   | burton  | XL
      3  | boots     | burton  | XL
(3 rows)
```

Видим, что все поля заполнены значениями, прописанными нами, и в **equip_id** отображается нумерация строк.

Модуль 7. Масштабируемость и отказоустойчивость

Задание: На основе созданной БД из Модуля 6 создаем кластер из двух pod в реплике конфигурации Active/Passive.

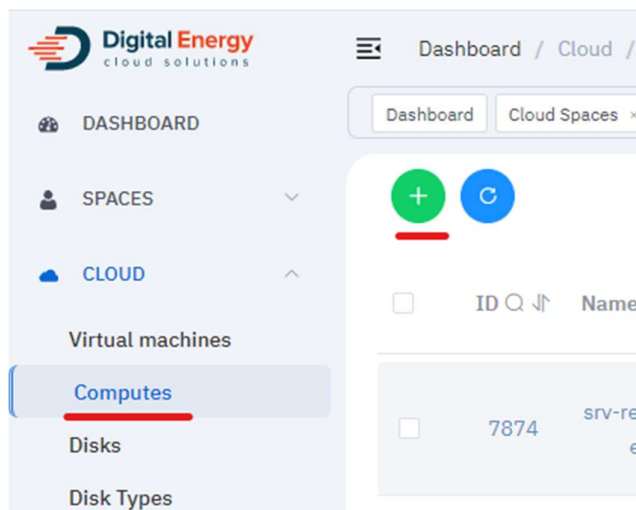
Уровень: 2

Продолжительность выполнения: 20 мин.

Создание двух VM test-node1 и test-node2

Выполнить последовательно два раза следующие действия:

Базис. Digital Energy – Computes



Указываем Account, Resource Group, Name, Architecture, Driver

×

Create a new Compute

1

2

3

General

Sizes

Network

*

Account

Universiade_education

▼

*

Resource Group

Test

▼

*

Name

Comp123

Description

*

Architecture

☒ X86_64

☐ PPC64_LE

*

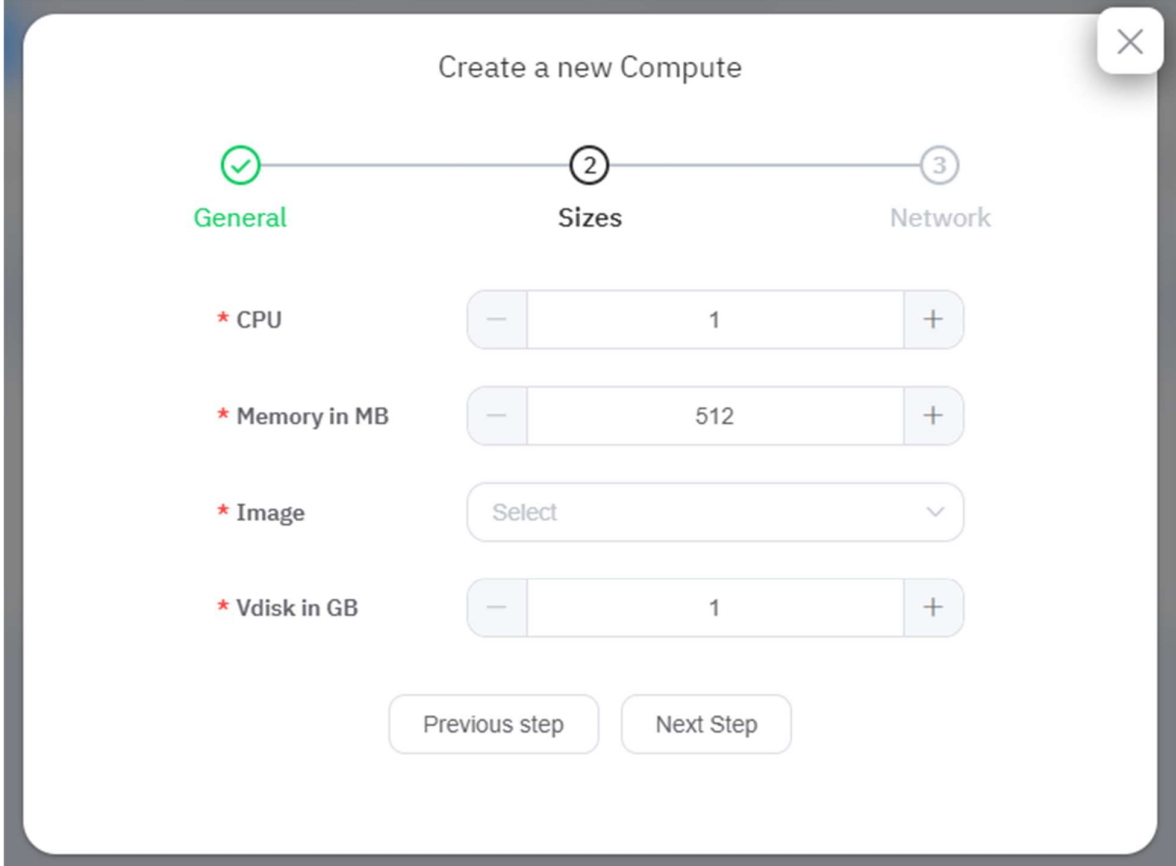
Driver

KVM_X86

▼

Next Step

Указываем CPU, Memory in MB, Image, Vdisk in GB



Create a new Compute

General Sizes Network

* CPU 1

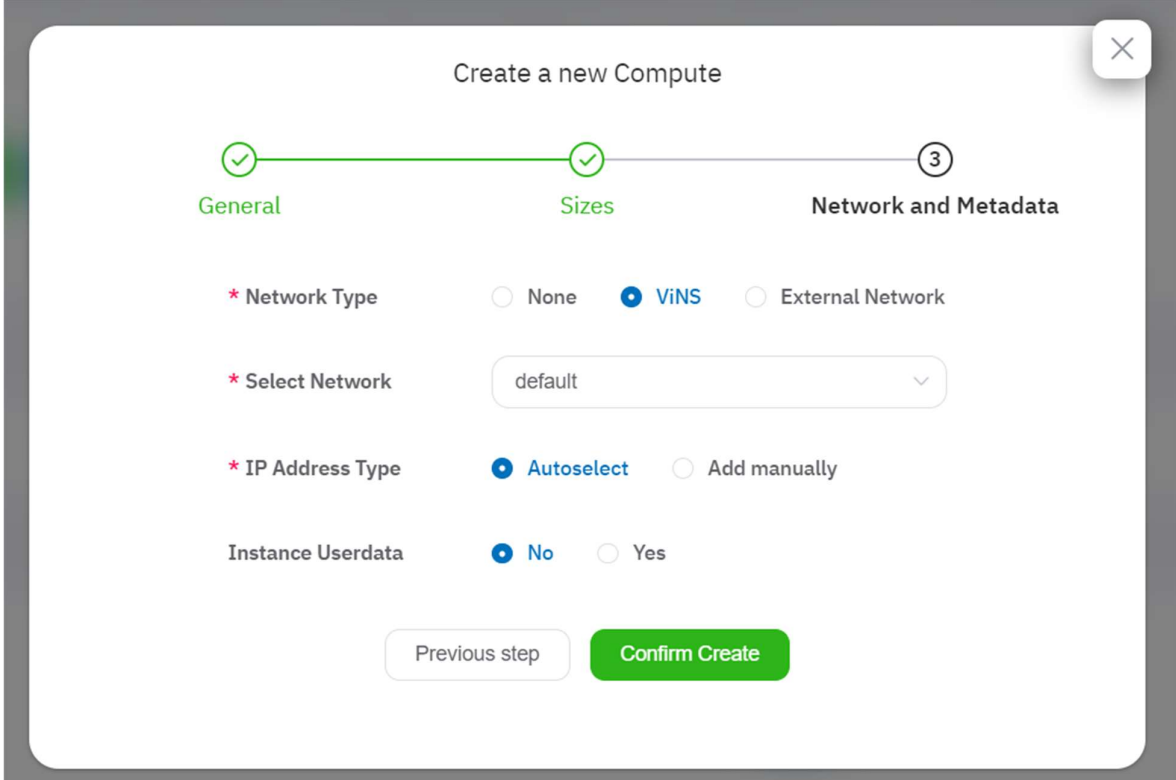
* Memory in MB 512

* Image Select

* Vdisk in GB 1

Previous step Next Step

Указываем Network Type



Create a new Compute

General Sizes Network and Metadata

* Network Type ☐ None ☒ ViNS ☐ External Network

* Select Network default

* IP Address Type ☒ Autoselect ☐ Add manually

Instance Userdata ☒ No ☐ Yes

Previous step Confirm Create

Нажимаем Confirm Creating

Active/Passive PostgreSQL Cluster с использованием Pacemaker, Corosync

Перед установкой

Для развёртывания кластера необходимо подготовить инфраструктуру, состоящую из минимум трёх узлов. Возможно использование как физических, так и виртуальных машин (как в примере ниже).

Узлы могут иметь произвольные названия. Здесь и ниже три узла называются node1, node2 и node3.

Выполните следующие действия:

1. В качестве узлов подготовьте три виртуальные машины одинаковой конфигурации и установите на них операционную систему Ubuntu 20.04 LTS.
2. Настройте синхронизацию времени между всеми узлами.
3. На всех узлах добавьте IP-адреса всех узлов в файл /etc/hosts.
4. Настроить обмен между узлами через SSH-ключи.

Установка ПО

На все подготовленные узлы кластера установите необходимое ПО при помощи команды:

```
$ sudo apt install corosync pcs pacemaker fence-agents postgresql
```

Поскольку пакет pacemaker содержит версию пакета resource-agents, которая не поддерживает работу с PostgreSQL версии 12, следует заменить его пакетом версии 4.7.0. Для этого:

1. Удалите resource-agents со всеми зависимостями:

```
$ sudo dpkg --remove --force-depends resource-agents
```

2. Скачайте и установите пакет resource-agents версии 4.7.0:

```
$ wget  
http://ftp.de.debian.org/debian/pool/main/r/resourceagents/resource-  
agents_4.7.0-1_amd64.deb  
$ sudo dpkg -i resource-agents_4.7.0-1_amd64.deb
```

Настройка кластера

Для первоначальной настройки кластера выполните следующие действия.

1. Измените пароль пользователя hacluster, который был создан автоматически на всех узлах кластера при установке пакетов :

```
$ sudo passwd hacluster
```

2. Произведите аутентификацию на одном из узлов кластера:

```
$ sudo pcs host auth node1 node2 node3
```

3. Создайте кластер под названием pgcluster:

```
$ sudo pcs cluster setup --force pgcluster node1 addr=ipaddr node2  
addr=ipaddr node3 addr=ipaddr
```

Параметры команды:

`ipaddr` – IP-адрес добавляемого узла;

`node1, node2, node3` – сетевые имена узлов кластера.

Внимание! IP-адреса всех узлов кластера должны быть явно указаны в файле `/etc/hosts` на всех узлах кластера.

В случае неудачной попытки создания кластера удалите все существующие файлы конфигурации кластера с помощью команды:

```
$ sudo pcs cluster destroy
```

5. Запустите кластер и настройте автоматический запуск кластера при запуске системы:

```
$ sudo systemctl enable corosync.service  
$ sudo systemctl enable pacemaker.service  
$ sudo pcs cluster enable -all $ sudo pcs cluster start -all
```

Эти действия завершают настройку кластера.

Чтобы узнать статус кластера используйте команду:

```
$ sudo pcs cluster status
```

Чтобы проверить синхронизацию узлов кластера используйте команду:

```
$ sudo corosync-cmapctl | grep members
```

Для отслеживания состояния кластера в реальном времени можно использовать команду:

```
$ sudo crm_mon -Afr
```

Настройка PostgreSQL

После первоначальной настройки кластера следует настроить СУБД PostgreSQL следующим образом.

1. Отключите запуск `postgresql.service` при загрузке системы на каждом узле кластера. Включать и отключать сервис при необходимости теперь будет Pacemaker.

```
$ sudo systemctl disable postgresql.service
```

2. На узле (в нашем случае это `node1`), который первоначально будет являться мастером, инициализируйте новую базу данных:

```
$ sudo -u postgres /usr/lib/postgresql/12/bin/initdb -D
/var/lib/postgresql/12/main
```

Если база уже запущена, то остановите процесс postgresql, очистите директорию `/var/lib/postgresql/12/main` и выполните команду ещё раз.

3. Запустите базу:

```
$ sudo -u postgres /usr/lib/postgresql/12/bin/pg_ctl -D
/var/lib/postgresql/12/main start
```

4. Установите пароль для пользователя postgres:

```
$ sudo -u postgres psql psql (12.5 (Ubuntu 12.5-0ubuntu0.20.04.1))
Type "help" for help. postgres=# alter user postgres password '12345';
ALTER ROLE
```

5. Создайте пользователя для репликации:

```
$ sudo -u postgres /usr/lib/postgresql/12/bin/createuser --replication
-P repl
```

6. Измените файл `/var/lib/postgresql/12/main/pg_hba.conf` и добавьте в него необходимые разрешения следующим образом:

#	TYPE	DATABASE	USER	ADDRESS	METHOD
#	"local"	is for Unix domain socket connections only			
local	all		all		trust
#	IPv4	local connections:			
host	all		all	127.0.0.1/32	trust
#	IPv6	local connections:			
host	all		all	:::1/128	trust
#Allow replication connections from localhost, by a user with the #replication privilege.					
local	replication		all		trust
host	replication		all	127.0.0.1/32	trust
host	replication		all	:::1/128	trust
host	replication		all	192.168.110.0/24	trust
host	all		all	192.168.110.0/24	trust

Внимание! В двух нижних строках в столбце ADDRESS следует указать вашу подсеть.

7. Измените файл `/var/lib/postgresql/12/main/postgresql.conf` и добавьте в него следующие строки:

```
listen_addresses = '*'
wal_level = replica
logging_collector = on
hot_standby = on
lc_messages = 'C'
wal_keep_segments = 10
```

8. Перезапустите СУБД PostgreSQL:

```
$ sudo -u postgres /usr/lib/postgresql/12/bin/pg_ctl -D  
/var/lib/postgresql/12/main stop  
$ sudo -u postgres /usr/lib/postgresql/12/bin/pg_ctl -D  
/var/lib/postgresql/12/main start
```

На других двух узлах (node2 и node3) выполните следующие действия:

1. Остановите процесс postgresql, если он запущен.
2. Очистите директорию /var/lib/postgresql/12/main.
3. Скопируйте базу данных с мастера (node1) при помощи команды:

```
$ sudo -u postgres pg_basebackup -D /var/lib/postgresql/12/main -h  
192.168.110.200 -X stream
```

Параметр 192.168.110.200 в данном случае – это IP-адрес узла node1, на котором мы произвели первоначальную настройку PostgreSQL и инициализировали базу данных.

Настройка ресурсов кластера

Создание ресурсов кластера

После настройки СУБД PostgreSQL на всех узлах кластера следует создать ресурсы кластера. Для этого выполните следующие действия:

1. Создайте ресурс с именем virtual_ip типа IPaddr2, который будет использоваться для подключения в базе данных PostgreSQL:

```
$ sudo pcs resource create virtual_ip IPaddr2 ip="192.168.110.204"  
cidr_netmask="24" meta migration-threshold="0" op monitor  
timeout="60s" interval="10s" on-fail="restart" op stop timeout="60s"  
interval="0s" on-fail="ignore" op start timeout="60s" interval="0s"  
on-fail="stop"
```

В качестве значения параметра ip укажите любой адрес из подсети, в которой находятся узлы кластера. Этот IP-адрес будет автоматически добавляться на интерфейс того узла, который будет являться мастером в тот момент времени.

2. Создайте ресурс с именем my-pgsql типа pgsql для управления конфигурацией PostgreSQL:

```
$ sudo pcs resource create my-pgsql pgsql  
pgctl="/usr/lib/postgresql/12/bin/pg_ctl"  
psql="/usr/lib/postgresql/12/bin/psql"  
pgdata="/var/lib/postgresql/12/main" rep_mode="sync" node_list="node1  
node2 node3" master_ip="192.168.110.204" restart_on_promote="false"  
check_wal_receiver="true" pgport="5432"  
primary_conninfo_opt="password=12345" repuser="repl"
```

3. Для созданного выше ресурса my-pgsql укажите, что он может иметь одно из нескольких состояний и менять их в зависимости от типа узла (master и slave):

```
$ sudo pcs resource promotable my-pgsql promoted-max=1 promotednode-  
max=1 clone-max=3 clone-node-max=1 notify=true
```

Свяжите два созданных выше ресурса, чтобы они запускались вместе на одном узле, и установите очерёдность запуска таким образом, чтобы ресурс `virtual_ip` запускался только после успешного запуска ресурса `my-pgsql`. Для этого создайте группу ресурсов `master-group` и добавьте в неё ресурсы:

```
$ sudo pcs resource group add master-group virtual_ip
$ sudo pcs constraint colocation add master-group with Master mypgsql-clone
$ sudo pcs constraint order promote my-pgsql-clone then start master-group symmetrical=false score=INFINITY
$ sudo pcs constraint order demote my-pgsql-clone then stop master-group symmetrical=false score=0
```

Создание ресурсов фенсинга

Для защиты разделяемых ресурсов и изоляции узла кластера при его неисправности существует механизм фенсинга (изоляции).

Чтобы вывести список доступных `fence-agents` используйте команду:

```
$ sudo pcs stonith list
```

При использовании виртуальных машин в качестве узлов кластера можно использовать агент `fence_virsh`. Чтобы вывести необходимые настройки для выбранного агента используйте команду:

```
$ sudo pcs stonith describe fence_virsh
```

Содержимое файла конфигурации PostgreSQL 12

`/var/lib/postgresql/12/main/postgresql.conf`

```
max_connections = 100
shared_buffers = 128M
dynamic_shared_memory_type = posix
max_wal_size = 1GB
min_wal_size = 80MB
log_timezone = 'Europe/Moscow'
datestyle = 'iso, mdy'
timezone = 'Europe/Moscow'
lc_messages = 'C.UTF-8'
lc_monetary = 'C.UTF-8'
lc_numeric = 'C.UTF-8'
lc_time = 'C.UTF-8'
default_text_search_config = 'pg_catalog.english'
listen_addresses = '*'
wal_level = replica
logging_collector = on
hot_standby = on
lc_messages = 'C'
wal_keep_segments = 10
```

Содержимое файла конфигурации PostgreSQL 12

/var/lib/postgresql/12/main/postgresql.conf

#	TYPE	DATABASE	USER	ADDRESS	METHOD
#	"local"	is for Unix domain socket connections only			
local	all		all		trust
#	IPv4 local connections:				
host	all		all	127.0.0.1/32	trust
#	IPv6 local connections:				
host	all		all	::1/128	trust
#	Allow replication connections from localhost, by a user with the				
#	replication privilege.				
Local	replication		all		trust
host	replication		all	127.0.0.1/32	trust
host	replication		all	::1/128	trust
host	replication		all	192.168.110.0/24	trust
host	all		all	192.168.110.0/24	trust

Содержимое файла конфигурации Corosync

/etc/corosync/corosync.conf

```
totem {
    version: 2
    cluster_name: pgcluster
    transport: knet
    crypto_cipher: aes256
    crypto_hash: sha256
}
nodelist {
    node {
        ring0_addr: 192.168 .110 .200
        name: node1
        nodeid: 1
    }
    node {
        ring0_addr: 192.168 .110 .201
        name: node2
        nodeid: 2
    }
    node {
        ring0_addr: 192.168 .110 .203
        name: node3
        nodeid: 3
    }
}
quorum {
    provider: corosync_votequorum
}
logging {
    to_logfile: yes
    logfile: /var/log/corosync/corosync.log
    to_syslog: yes
    timestamp: on
}
```

Модуль 8. Мониторинг, логирование и оповещение событий

Задание: Установка и развертывание Prometheus+Grafana+Node_exporter

Настройка метрик нагрузки вашей ОС с отображением в Grafana

Уровень: 3

Продолжительность выполнения: 45мин.

Установите и запустите node_exporter на узле

Мы собираемся собирать метрики с узла Linux с помощью [node_exporter](#).

Загрузите сжатый пакет node_exporter.

```
wget
https://github.com/prometheus/node_exporter/releases/download/v*/node_
exporter-*.*-amd64.tar.gz
```

Извлеките двоичный файл node_exporter, который будет собирать метрики с компьютера Linux, форматировать эти метрики в формате Prometheus и передавать эти метрики на внутренний порт для очистки Prometheus.

```
tar xvfz node_exporter-*.*-amd64.tar.gz
```

Перейдите в каталог, созданный во время извлечения.

```
cd node_exporter-*.*-amd64
```

Сделайте двоичный файл node_exporter исполняемым.

```
chmod +x node_exporter
```

Запустите бинарный файл node_exporter.

```
./node_exporter
```

Вы должны увидеть вывод, подобный этому, указывающий, что Node Exporter теперь работает и предоставляет метрики на порту 9100:

```
INFO[0000] Starting node_exporter (version=0.16.0, branch=HEAD,
revision=d42bd70f4363dced6b77d8fc311ea57b63387e4f) source="node_exporter.go:82"
INFO[0000] Build context (go=go1.9.6, user=root@a67a9bc13a69, date=20180515-15:53:28)
source="node_exporter.go:83"
INFO[0000] Enabled collectors:                                source="node_exporter.go:90"
INFO[0000]   - boottime                                       source="node_exporter.go:97"
...
INFO[0000] Listening on :9100                                source="node_exporter.go:111"
```

Проверьте, что метрики экспортируются через порт 9100.

```
curl http://localhost:9100/metrics
```

Вы должны увидеть такой вывод:

```
# HELP go_gc_duration_seconds A summary of the GC invocation
durations.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 3.8996e-05
go_gc_duration_seconds{quantile="0.25"} 4.5926e-05
go_gc_duration_seconds{quantile="0.5"} 5.846e-05
# etc.
```

Если вы видите метрики на своем экране, все в порядке. Если нет, проверьте свои шаги на наличие опечаток, убедитесь, что бинарный файл является исполняемым, и работает ли curl с другими URL-адресами.

Если вы не хотите запускать node_exporter напрямую из командной строки, вы можете создать для него службу systemd по аналогии с [Созданием службы systemd для управления агентом](#).

Установите Prometheus

Затем мы загрузим и установим [Prometheus](#) на узел, чтобы собрать метрики, предоставленные node_exporter, и отправить их в Grafana. Prometheus собирает метрики и делает их доступными для поиска и визуализации. Обычно Prometheus извлекает метрики из настроенных источников, но мы будем использовать remote_write функцию, которая будет передавать метрики из Prometheus в Grafana.

Загрузите архив пакета Prometheus.

```
wget
https://github.com/prometheus/prometheus/releases/download/v*/promethe
us-*. *-amd64.tar.gz
```

Извлеките двоичный файл.

```
tar xvf prometheus-*. *-amd64.tar.gz
```

Перейдите в каталог, созданный во время извлечения.

```
cd prometheus-*. *
```

Настройка экземпляров Prometheus

Ваш локально работающий экземпляр Prometheus должен быть правильно настроен для доступа к метрикам Node Exporter. Следующий [prometheus.yml](#) пример файла конфигурации сообщает экземпляру Prometheus о том, как часто выполнять очистку от Node Exporter через localhost:9100:

```
global:
  scrape_interval: 15s

scrape_configs:
- job_name: node
  static_configs:
    - targets: ['localhost:9100']
```

Запустить Prometheus, используя флаг, --config.file указывающий на конфигурацию Prometheus, которую вы создали выше :

```
./prometheus --config.file=./prometheus.yml
```

Изучение метрик Node Exporter через браузер выражений Prometheus

Теперь, когда Prometheus извлекает метрики из работающего экземпляра Node Exporter, вы можете исследовать эти метрики с помощью пользовательского интерфейса Prometheus (он же обозреватель [выражений](#)). Перейдите localhost:9090/graph в браузере и используйте основную панель выражений в верхней части страницы для ввода выражений. Панель выражений выглядит так:

Метрики, специфичные для Node Exporter, имеют префикс `node_` и включают такие метрики, как `node_cpu_seconds_total` и `node_exporter_build_info`.

Нажмите на ссылки ниже, чтобы увидеть некоторые примеры показателей:

Метрика	Значение
rate(node_cpu_seconds_total{mode="system"}[1m])	Среднее время ЦП, проведенное в системном режиме, в секунду за последнюю минуту (в секундах)
node_filesystem_avail_bytes	Пространство файловой системы, доступное пользователям без полномочий root (в байтах)
rate(node_network_receive_bytes_total[1m])	Средний сетевой трафик, полученный в секунду за последнюю минуту (в байтах)

Установить на Debian или Ubuntu

1. Загрузите и установите

Вы можете установить Grafana, используя официальный репозиторий APT, загрузив пакет `.deb` или загрузив двоичный `.tar.gz` файл.

Установить из репозитория APT

Если вы устанавливаете из репозитория APT, то Grafana автоматически обновляется каждый раз, когда вы запускаете `apt-get update`.

Чтобы установить последнюю версию:

```
sudo apt-get install -y apt-transport-https
sudo apt-get install -y software-properties-common wget
sudo wget -q -O /usr/share/keyrings/grafana.key
https://apt.grafana.com/gpg.key
```

Добавьте этот репозиторий для стабильных выпусков:

```
echo "deb [signed-by=/usr/share/keyrings/grafana.key]
https://apt.grafana.com stable main" | sudo tee -a
/etc/apt/sources.list.d/grafana.list
```

Добавьте этот репозиторий, если вам нужны бета-версии:

```
echo "deb [signed-by=/usr/share/keyrings/grafana.key]
https://apt.grafana.com beta main" | sudo tee -a
/etc/apt/sources.list.d/grafana.list
```

После добавления репозитория:

```
sudo apt-get update

# Install the latest OSS release:
sudo apt-get install grafana

# Install the latest Enterprise release:
sudo apt-get install grafana-enterprise
```

Установите пакет .deb

Если вы устанавливаете `.deb` пакет, вам нужно будет вручную обновлять Grafana для каждой новой версии.

1. На [странице загрузки Grafana](#) выберите версию Grafana, которую хотите установить.
 - По умолчанию выбирается самая последняя версия Grafana.
 - В поле **Version** отображаются только готовые версии. Если вы хотите установить бета-версию, нажмите «**Nightly Builds**», а затем выберите версию.
2. Выберите **Edition**.
 - **Enterprise** — рекомендуемая загрузка. Функционально идентичен версии с открытым исходным кодом, но включает функции, которые вы можете разблокировать с помощью лицензии, если вы того пожелаете.
 - **Open Source** — функционально идентичен версии Enterprise, но вам нужно будет загрузить версию Enterprise, если вам нужны функции Enterprise.
3. В зависимости от используемой системы выберите **Linux** или **ARM**.
4. Скопируйте и вставьте код со страницы установки в командную строку и запустите. Это следует схеме, показанной ниже.

```
sudo apt-get install -y adduser
wget <.deb package url>
sudo dpkg -i grafana<edition>_<version>_amd64.deb
```

Установить из бинарного файла .tar.gz

Загрузите последний [.tar.gz файл](#) и распакуйте его. Файлы извлекаются в папку, названную в честь загруженной версии Grafana. Эта папка содержит все файлы, необходимые для запуска Grafana. В этом пакете нет сценариев инициализации или сценариев установки.

```
wget <tar.gz package url>
sudo tar -zxvf <tar.gz package>
```

2. Запустите сервер

Это запустит `grafana-server` процесс от имени `grafana` пользователя, который был создан во время установки пакета.

Если вы установили с АРТ репозиторием или `.deb` пакетом, вы можете запустить сервер, используя `systemd` или `init.d`. Если вы установили бинарный `.tar.gz` файл, то вам нужно выполнить бинарник.

Запустите сервер с помощью `systemd`

Чтобы запустить службу и убедиться, что служба запущена:

```
sudo systemctl daemon-reload
sudo systemctl start grafana-server
sudo systemctl status grafana-server
```

Настройте сервер Grafana для запуска при загрузке:

```
sudo systemctl enable grafana-server.service
```

Войдите в Grafana

Чтобы войти в Grafana в первый раз:

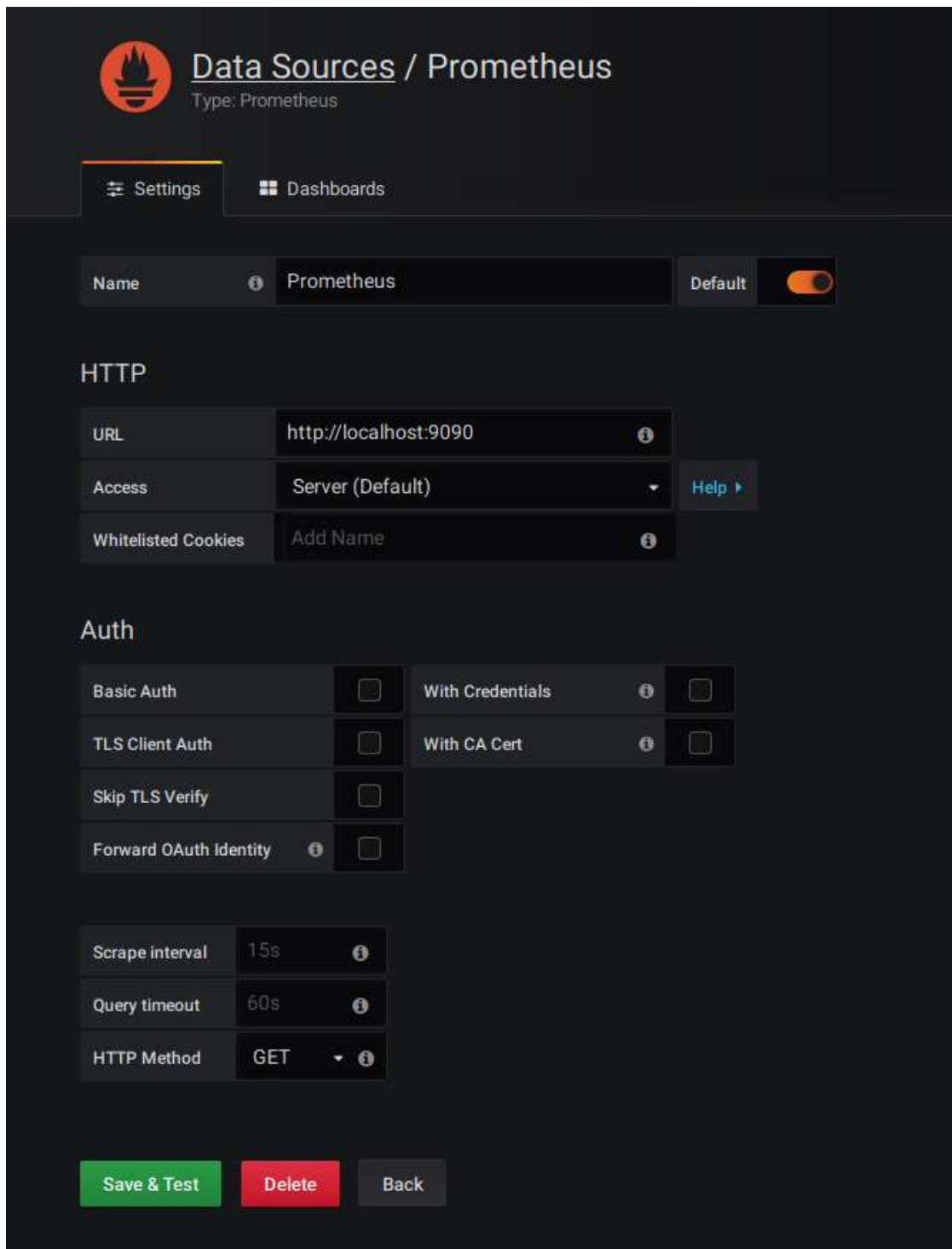
1. Откройте веб-браузер и перейдите по адресу `http://localhost:3000/`.
HTTP-порт по умолчанию, который слушает Grafana, — это 3000, если вы не настроили другой порт.
2. На странице входа введите `admin` в качестве имени пользователя и пароль.
3. Щелкните **Sign in**.
В случае успеха вы увидите запрос на смену пароля.
4. Нажмите **OK** в приглашении и измените свой пароль.

Настройка Grafana

После установки всех компонент можно приступить к настройке Grafana.

Источник данных (datasource)

Первый шаг — настройка источника данных (datasource). По сути, это источник метрик для Grafana. Мы будем получать метрики с нашего сервера Prometheus, поэтому в качестве источника данных выбираем Prometheus и указываем URL `http://localhost:9090`. Остальные параметры оставляем по умолчанию.



Data Sources / Prometheus
Type: Prometheus

Settings Dashboards

Name Prometheus Default ☒

HTTP

URL http://localhost:9090

Access Server (Default) Help

Whitelisted Cookies Add Name

Auth

Basic Auth ☐ With Credentials ☐

TLS Client Auth ☐ With CA Cert ☐

Skip TLS Verify ☐

Forward OAuth Identity ☐

Scrape interval 15s

Query timeout 60s

HTTP Method GET

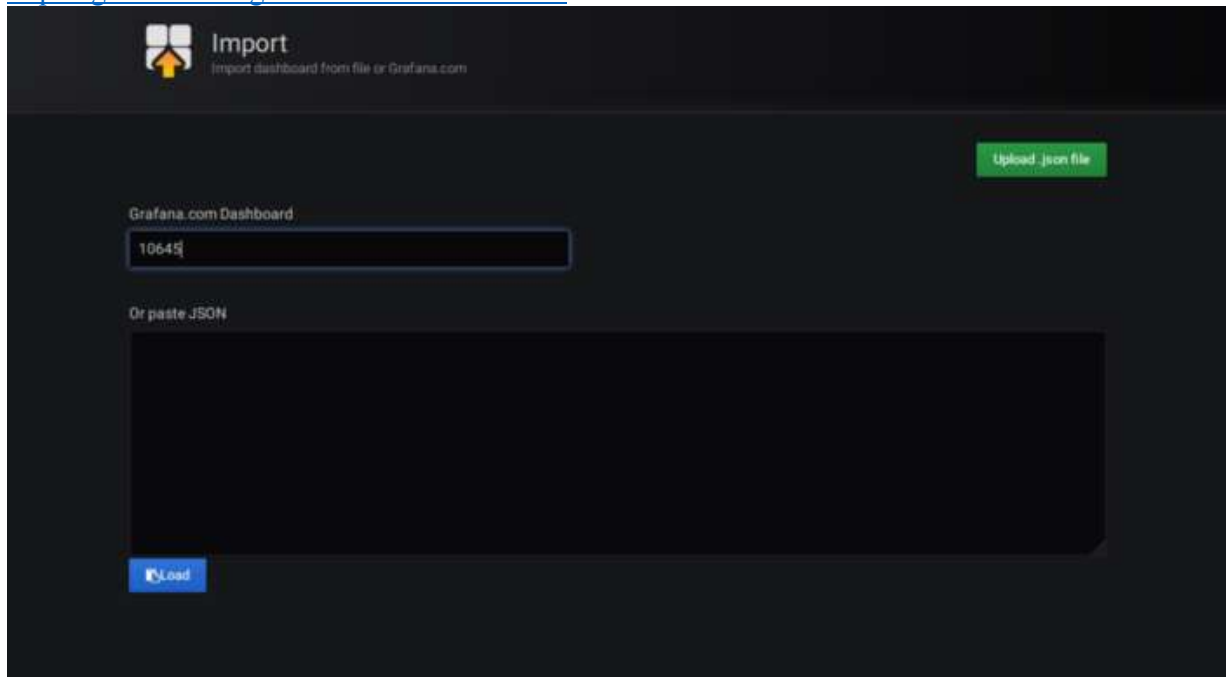
Save & Test Delete Back

Дашборд (dashboard)

После настройки Datasource, мы можем создать дашборд. Дашборд — это просто набор панелей, расположенных на одной странице. Панели бывают разные: от простого текста до круговых диаграмм. Каждую панель можно настроить для отображения различных метрик.

Создание своего дашборда с нуля может быть нелегкой задачей. К счастью, Grafana предоставляет простой способ импорта дашбордов, созданных другими пользователями. После импорта дашборд можно настроить в соответствии с вашими потребностями. Для примера я будем использовать готовый дашборд, созданный "cordobatus" и расположенный по следующей ссылке:

<https://grafana.com/grafana/dashboards/10795>



Чтобы установить дашборд, укажите его ID на странице Import.

После импорта откройте установленный дашборд с помощью выпадающего списка в левом верхнем углу. При первом открытии он выглядит следующим образом.



Плагины

Хотя плагины не являются обязательными, но наш дашборд содержит панель, которая использует сторонний плагин, отсутствующий по умолчанию. Эту панель можно удалить (возможно, заменив ее чем-то другим), либо установить недостающий плагин.

Необходимый нам плагин предназначен для отображения круговых диаграмм. Его можно найти по ссылке ниже.

<https://grafana.com/grafana/plugins/grafana-piechart-panel>

Установка плагина описана на его странице. Откройте командную строку и выполните следующую команду:

```
grafana-cli plugins install grafana-piechart-panel
```

Для завершения установки плагина перезапустите Grafana:

```
service grafana-server restart
```

Далее можно обновить страницу — панель с круговой диаграммой должна отображаться правильно.

Заключение

После установки всех компонент и настройки Grafana у вас должно получиться что-то вроде этого:



Взглянув на эту страницу, вы можете получить информацию о состоянии сервера: начиная с того, как долго он работает, заканчивая загрузкой процессора и использованием памяти.

Модуль 9. Виртуализация в DevOps

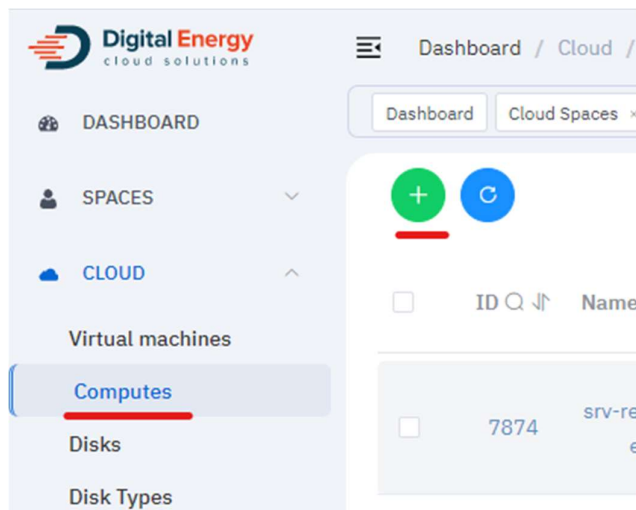
Задание: Развертывание ВМ в Базисе на базе шаблона Ubuntu20.04 LTS

Установка на нее пакетов: vim, mc, net-tools

Уровень: 2

Продолжительность выполнения: 10мин.

Базис. Digital Energy – Computes



Указываем Account, Resource Group, Name, Architecture, **Driver**

×

Create a new Compute

①

②

③

General

Sizes

Network

* Account

Universiade_education

▼

* Resource Group

Test

▼

* Name

Comp123

Description

* Architecture

☒ X86_64

☐ PPC64_LE

* Driver

KVM_X86

▼

Next Step

Указываем CPU, Memory in MB, Image, Vdisk in GB

Create a new Compute

General

Sizes

Network

* CPU

—

1

+

* Memory in MB

—

512

+

* Image

Select

▼

* Vdisk in GB

—

1

+

Previous step

Next Step

Доступные для использования Images

Digital Energy

cloud solutions

Dashboard / Cloud / Images

alexey_e

DASHBOARD

SPACES

CLOUD

Virtual machines

Computes

Disks

Disk Types

Images

NETWORK

MANAGED SERVICES

SYSTEM

Recycle Bin

Audits

Dashboard

Cloud Spaces

Resource Groups

Virtual machines

Computes

Images

Template Images

CD-ROM Images

Virtual Images

	ID	Name	Status	Architecture	Type	Size	Pub
☐	441	k8ci-alt8sp-k8ci-v1.22.5-tea	CREATED	X86_64	linux	32 GiB	Yes
☐	435	Astra Linux CE	CREATED	X86_64	linux	5 GiB	Yes
☐	431	T-Pot	CREATED	X86_64	linux	1 GiB	Yes
☐	425	k8s-1.22.3_u20_ccm_v1.2	CREATED	X86_64	linux	5 GiB	Yes
☐	423	k8s-1.22.3_u20_ccm	CREATED	X86_64	linux	5 GiB	Yes
☐	417	rocky8-jul-20	CREATED	X86_64	linux	4 GiB	No

Указываем Network Type

Create a new Compute

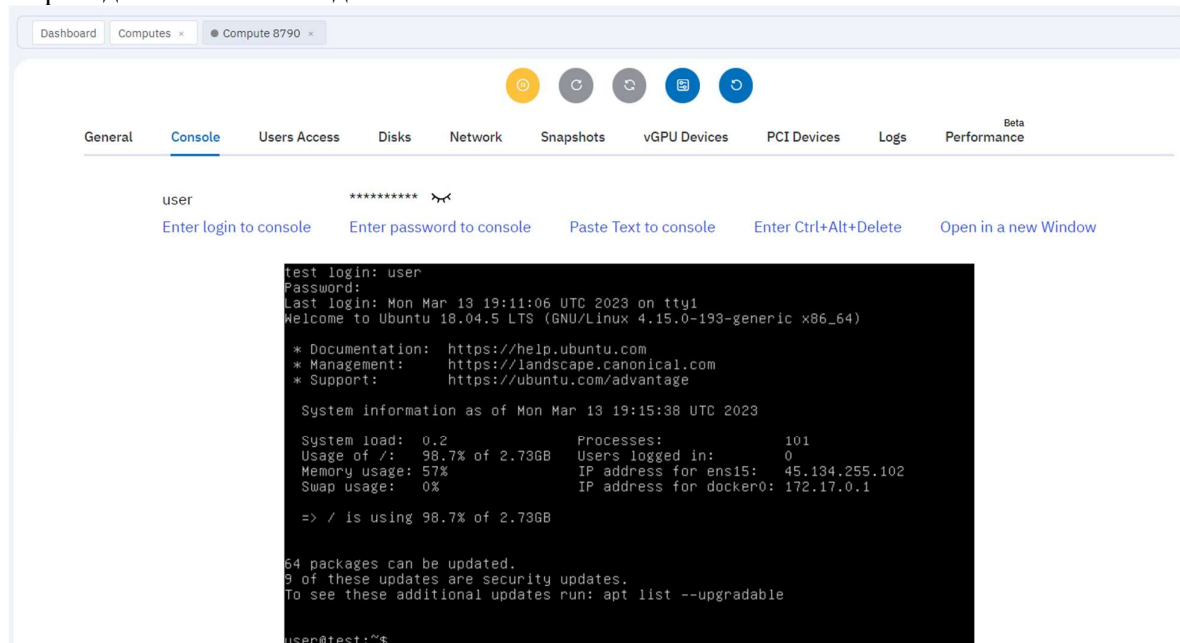
General Sizes Network

* Network Type ☒ None ☐ ViNS ☐ External Network

Previous step Confirm Create

Нажимаем Confirm Creating

Переходим в консоль созданной VM



Сначала откройте терминальное приложение, а затем обновите базу данных пакетов с помощью [команды apt](#) или [команды apt-get](#) :

```
$ sudo apt update
```

Установка Vim, Mc, Net-tools

```
$ sudo apt -y install vim
$ vim --version
```

```
$ sudo apt -y install mc
```

```
$ sudo apt -y install net-tools
```

Модуль 11. Работа с Terraform: как управлять облачной инфраструктурой

Задание: Развёртывание ВМ в БАЗИС на базе шаблона Ubuntu20.04 LTS с использованием Terraform* + provider для БАЗИСА

Уровень: 5

Продолжительность выполнения: 30 мин.

Для выполнения этого задания нужно предоставить для скачивания на локальный компьютер дистрибутив Terraform

Установка Terraform провайдера DECORT

Начало

Старт возможен по двум путям:

1. Установка через собранные пакеты.
2. Ручная установка.

Установка через собранные пакеты.

1. Скачайте и установите terraform по ссылке: <https://learn.hashicorp.com/tutorials/terraform/install-cli?in=terraform/aws-get-started>
2. Создайте файл `main.tf` и добавьте в него следующий блок.

```
provider "decort" {  
  authenticator = "oauth2"  
  #controller_url = <DECORT_CONTROLLER_URL>  
  controller_url = "https://dsl.digitalenergy.online"  
  #oauth2_url = <DECORT_SSO_URL>  
  oauth2_url     = "https://sso.digitalenergy.online"  
  allow_unverified_ssl = true  
}
```

3. Выполните команду

```
terraform init
```

Провайдер автоматически будет установлен на ваш компьютер из terraform registry.

Ручная установка

1. Скачайте и установите Go по ссылке: <https://go.dev/dl/>
2. Скачайте и установите terraform по ссылке: <https://learn.hashicorp.com/tutorials/terraform/install-cli?in=terraform/aws-get-started>
3. Склонируйте репозиторий с провайдером, выполнив команду:

```
git clone https://github.com/rudecs/terraform-provider-decort.git
```

4. Перейдите в скачанную папку с провайдером и выполните команду

```
go build -o terraform-provider-decort
```

Если вы знаете как устроен *makefile*, то можно изменить в файле Makefile параметры под вашу ОС и выполнить команду

```
make build
```

5. Полученный файл необходимо поместить:

Linux:

```
~/terraform.d/plugins/${host_name}/${namespace}/${type}/${version}/${target}
```

Windows:

```
%APPDATA%\terraform.d\plugins\${host_name}\${namespace}\${type}\${version}\${target}
```

ВНИМАНИЕ: для ОС Windows %APP_DATA% является каталогом, в котором будут помещены будущие файлы terraform. Где:

- host_name - имя хоста, держателя провайдера, например, digitalenergy.online
- namespace - пространство имен хоста, например, decort
- type - тип провайдера, может совпадать с пространством имен, например, decort
- version - версия провайдера, например, 1.2
- target - версия ОС, например, windows_amd64

6. После этого, создайте файл main.tf.

7. Добавьте в него следующий блок

```
terraform {
  required_providers {
    decort = {
      version = "1.2"
      source  = "digitalenergy.online/decort/decort"
    }
  }
}
```

В поле version указывается версия провайдера.

Обязательный параметр

Тип поля - строка

ВНИМАНИЕ: Версии в блоке и в репозитории, в который был помещен провайдер должны совпадать!

В поле source помещается путь до репозитория с версией вида:

```
${host_name}/${namespace}/${type}
```

ВНИМАНИЕ: все параметры должны совпадать с путем репозитория, в котором помещен провайдер.

8. В консоле выполнить команду

```
terraform init
```

9. Если все прошло хорошо - ошибок не будет.

Более подробно о сборке провайдера можно найти по ссылке: <https://learn.hashicorp.com/tutorials/terraform/provider-use?in=terraform/providers>

Инициализация Terraform провайдера DECORT

Список аргументов для инициализации

Перед началом использования любой Terraform провайдер должен быть инициализирован.

В процессе инициализации Terraform провайдера DECORT проверяется корректность переданных аргументов и выполняется авторизация в указанном контроллере облачной инфраструктуры. Подробнее о способах авторизации в платформе DECORT смотри соответствующий [раздел](#).

При инициализации Terraform провайдера DECORT используются следующие аргументы:

Аргумент	Переменная окружения	Описание
allow_unverified_ssl	-	Если данный аргумент явно установлен в true, то провайдер не будет проверять SSL сертификаты при взаимодействии с авторизационным сервисом OAuth2 и контроллером облачной платформы. Отключение проверок может быть полезным при работе в доверенной среде, использующей самоподписанные SSL сертификаты. Однако, так как отключение проверок несёт потенциальные риски безопасности, данную настройку следует использовать с осторожностью. Разрешённые значения: false (значение по умолчанию) и true.
app_id	DECORT_APP_ID	Идентификатор приложения (клиента) для авторизации в контроллере облачной платформы в режиме oauth2. Аргументы app_id и app_secret являются обязательными для режима авторизации authenticator=oauth2. Если app_id не задан в tf-файле, то провайдер

		будет использовать значение из переменной окружения DECORT_APP_ID.
app_secret	DECORT_APP_SECRET	Секретный код приложения (клиента) для авторизации в контроллере облачной платформы в режиме oauth2. Аргументы app_id и app_secret являются обязательными для режима авторизации authenticator=oauth2. Если app_secret не задан в tf-файле, то провайдер будет использовать значение из переменной окружения DECORT_APP_SECRET.
authenticator	-	Режим авторизации при подключении к контроллеру облачной платформы. Доступные режимы: oauth2, legacy или jwt. Данный аргумент является обязательным.
controller_url	-	URL контроллера облачной платформы, через который будет осуществляться управление облачными ресурсами. Данный аргумент является обязательным.
jwt	DECORT_JWT	JSON Web Token (JWT), который используется для авторизации в контроллере облачной платформы в режиме jwt. Данный аргумент является обязательным для режима авторизации authenticator=jwt. Если jwt не задан в tf-файле, то провайдер будет использовать значение из переменной окружения DECORT_JWT
oauth2_url	DECORT_OAUTH2_URL	URL авторизационного сервиса OAuth2, который используется для управления доступом пользователей (или программных клиентов) к контроллеру облачной платформы. Данный аргумент является обязательным для режимов авторизации authenticator=oauth2 или authenticator=jwt. Если oauth2_url не задан в tf-файле, то провайдер будет использовать значение из переменной окружения DECORT_OAUTH2_URL
password	DECORT_PASSWORD	Пароль для авторизации в контроллере облачной платформы в режиме legacy. Аргументы password и user являются обязательными для режима авторизации authenticator=legacy. Если password не задан в tf-файле, то провайдер будет использовать значение из переменной окружения DECORT_PASSWORD.
user	DECORT_USER	Имя пользователя для авторизации в контроллере облачной платформы в режиме legacy.

		Аргументы user и password являются обязательными для режима авторизации authenticator=legacy. Если user не задан в tf-файле, то провайдер будет использовать значение из переменной окружения DECORT_USER.
--	--	---

Пример инициализации

Пример инициализации Terraform провайдера DECORT:

```
provider "decort" {
  authenticator = "oauth2"
  controller_url = "https://ctrl.decort.online"
  oauth2_url = "https://oauth2.decort.online:7777"
}
```

В данном примере используется режим авторизации oauth2.

Как отмечено выше, в данном режиме требуется указать аргументы app_id и app_secret, идентифицирующие пользователя (или приложение-клиент), от лица которого будут выполняться дальнейшие действия. Однако, так как данная информация является конфиденциальной (по сути, она эквивалентна паре *имя пользователя : пароль*), то в общем случае заносить такого рода данные в tf-файл не следует. Рекомендуется определять в среде запуска Terraform переменные окружения DECORT_APP_ID и DECORT_APP_SECRET, из которых провайдер извлечёт нужные данные. Приведенный пример подразумевает, что нужная информация будет получена из этих переменных окружения.

Пользователь, от лица которого Terraform будет выполнять действия в облачной платформе, должен заранее создать пару *Application ID* и *Application Secret* в авторизационном приложении OAuth2. Именно эти значения, а также URL авторизационного приложения OAuth2, должны присваиваться аргументам app_id, app_secret и oauth2_url соответственно для успешной инициализации провайдера.

Также обратите внимание на формат задания аргументов controller_url и oauth2_url. В общем случае они должны содержать идентификатор протокола (*https://*) и сетевой порт, если он отличается от порта по умолчанию (в примере для авторизационного сервиса OAuth2 указан порт 7777). Эту информацию вы можете узнать у администратора вашей облачной инфраструктуры DECORT.

Пример работы

Ниже приведён подробно откомментированный пример, показывающий, как создать виртуальный сервер (aka *compute* на базе системы виртуализации KVM x86) в облачной платформе DECORT с помощью соответствующего Terraform провайдера. Сервер будет создан в новой ресурсной группе, к нему будет подключён один предварительно созданный диск, у сервера будет прямое сетевое подключение во внешнюю сеть.

Идентификатор образа операционной системы, на базе которого должен быть создан виртуальный сервер, считывается из облачной платформы с помощью *source data* функции [decort_image]([link to follow](#)).

Далее мы с помощью *resource* функции [decort_resgroup]([link to follow](#)) создаём новую ресурсную группу, в которую будет помещён этот виртуальный сервер. В качестве альтернативы, для получения информации об уже имеющейся ресурсной группе можно использовать *source data* функцию с таким же названием.

Затем с помощью *resource* функции [decort_disk]([link to follow](#)) создаётся диск, который будет подключён к виртуальному серверу в качестве дополнительного. Помимо этого дополнительного диска у сервера будет также и загрузочный диск, на который в процессе создания сервера клонируется выбранный образ операционной системы.

Виртуальный сервер - в данном примере на базе системы виртуализации KVM x86 - создаётся посредством *resource* функции [decort_kvmvm]([link to follow](#)).

Только авторизованные в контроллере облачной платформы пользователи могут управлять облачными ресурсами. Подробнее о способах авторизации см. [Обзор облачной платформы DECORT](#).

```
# 1. Initialize DECORT plugin and connection to DECORT cloud controller
# NOTE: in this example credentials are expected to come from
# DECORT_APP_ID and DECORT_APP_SECRET environmental variables - set them
# in the shell before calling terraform.
# Alternatively you may define plugin parameters app_id and app_secret in
# the TF file, however, this may be insecure if you plan to share this TF
# file with others.
provider "decort" {
  authenticator = "oauth2"
  controller_url = "<<DECORT_CONTROLLER_URL>>" # specify correct DECORT
controller URL, e.g. "https://dsl.digitalenergy.online"
  oauth2_url = "<<DECORT_SSO_URL>>" # specify corresponding DECORT SSO
URL, e.g. "https://sso.digitalenergy.online"
  # allow_unverified_ssl = true
}

# 2. Load account to use - new VM will belong to this account
data "decort_account" "my_account" {
  name = "<<NAME_OF_THE_ACCOUNT_TO_USE>>" # Specify the name of one of
your accounts
}

# 3. Load OS image to use for VM deployment
data "decort_image" "os_image" {
  image_id = "<<OS_IMAGE_ID>>" # Specify OS image id, e.g. 1234. You can
get accessible image id from data "decort_image_list"
```

```
}

# 4. Create new Resource Group in the selected account, new VM will be
created in this RG
resource "decort_resgroup" "my_rg" {
  name = "NewRgByTF"
  account_id = data.decort_account.my_account.id
  # if you want to set resource quota on this Resource Group, uncomment
  # the following code fragment
  # quota {
  #   cpu = 8      # CPU limit
  #   ram = 8912   # RAM limit in MB
  #   disk = 96    # disk volume limit in GB
  # }
}

# 5. Create extra disk, which will be attached to the new VM.
# This step is optional - if you do not want extra disks on your VM, skip
it
# and comment out extra_disks parameter when creating VM below.
resource "decort_disk" "extra_disk" {
  name = "extra-disk-for-vm"
  account_id = data.decort_account.my_account.id
  size = 5      # disk size in GB
  type = "D"    # disk type, always use "D" for extra disks
  sep_id = data.decort_image.os_image.sep_id # use the same SEP ID as the
OS image
  pool = "<<DATA_POOL_NAME>>" # consult your DECORT platform admin for
configured storage pool names
}

# 6. Create virtual machine (a compute of type KVM VM x86 in this example)
# Now that we have all necessary components at hand, we may create a
virtual machine.
# This VM will be based on the previously obtained OS image, located in
the specified
# Resource Group, directly connected to an external network, have a boot
disk of
# specified size and one extra disk attached.
resource "decort_kvmvm" "my_new_vm" {
  name = "tf-managed-vm"
  rg_id = decort_resgroup.my_rg.id
  arch = "KVM_X86"      # "KVM_PPC" for IBM Power or "KVM_X86" for Intel
```

```
cpu = 1                # CPU count
ram = 1024              # RAM size in MB, must be even number, ideally a
power of 2
boot_disk_size = 10    # Boot disk size in GB
image_id = data.decort_image.os_image.id
description = "Test KVM VM Compute managed by Terraform"
extra_disks = [ decort_disk.extra_disk.id ]

network {
  net_type = "EXTNET"
  net_id = <<EXT_NET_ID>> # specify external network ID to use,
consult your DECORT platform admin for correct IDs
  # ip_address = "<<SOME VALID AND FREE IP ADDRESS>>" # you may
optionally request a specific IP address
}
```

Модуль 12. Конфигурационное управление. Что такое IaC

Задание: Создание 2 VM на базе шаблона CentOS 7.9, установка на них ansible и настройка. Создание ansible-playbook по созданию 2 папок, 1 пользователя, установки пакетов vim, mc с 1 VM на 2 VM

Уровень: 5

Продолжительность выполнения: 45 мин.

Установка Ansible

Чтобы начать использовать Ansible в качестве средства управления серверной инфраструктурой, вам необходимо установить программное обеспечение Ansible на машину, которая будет служить управляющим узлом Ansible.

На своем узле управления выполните следующую команду, чтобы включить PPA (личный архив пакетов) официального проекта в список источников вашей системы:

```
$ sudo apt-add-repository ppa:ansible/ansible
```

Нажмите ENTER, когда будет предложено принять добавление PPA.

Затем обновите индекс пакетов вашей системы, чтобы он знал о пакетах, доступных во вновь включенном PPA:

```
$ sudo apt update
```

После этого обновления вы можете установить программное обеспечение Ansible с помощью:

```
$ sudo apt install ansible
```

Ваш управляющий узел Ansible теперь имеет все программное обеспечение, необходимое для администрирования ваших хостов. Далее мы рассмотрим, как добавить ваши хосты в файл инвентаризации узла управления, чтобы он мог ими управлять.

Проверяем установленную версию:

```
$ ansible --version
ansible [core 2.12.10]
  config file = /etc/ansible/ansible.cfg
  configured module search path =
  ['/home/user/.ansible/plugins/modules',
  '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-
  packages/ansible
  ansible collection location =
  /home/user/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.8.10 (default, Nov 14 2022, 12:59:47) [GCC 9.4.0]
  jinja version = 2.10.1
  libyaml = True
```

Давайте посмотрим на конфигурационный файл

```
vim /etc/ansible/ansible.cfg
```

здесь укажем путь до inventory файла

раскомментируем

```
inventory      = /etc/ansible/hosts
```

раскомментируем

```
host_key_checking = False
```

не будем проверят рукопожатие

теперь перейдем в inventory файл

```
vim /etc/ansible/hosts
```

запишем сюда адреса наших машин

```
node1 ansible_host=host ip1
node2 ansible_host=host ip 2
```

проверим доступность наших машин

```
ansible all -m ping
```

Что произошло? как видно из сообщения нам не хватает прав т.е. как мы знаем для доступа по ssh нам нужен логин и пароль, либо ключ

давайте зададим пароль

```
node1 ansible_host=178.154.198.76 ansible_user=ansible
ansible_password=ansible
node2 ansible_host=178.154.198.205
```

пробуем еще видим "msg": "Failed to connect to the host via ssh: Permission denied"

Ура, нам удалось подключиться к одной из удаленных машин

Давайте для второй машины создадим нового пользователя

```
adduser ansible
```

теперь добавим его в файл конфигурации

```
ansible_user=ansible ansible_password=ansible
```

проверяем, успех! теперь у нас подключено 2 машины

Не забываем убрать все лишнее из /etc/ansible/hosts и добавить параметр для уз ansible в visudo NOPASSWD:ALL

В папку group_vars сделаем playbook

```
test-playbook.yaml:
- name: Test
  hosts: all
  become: yes

  tasks:

  - name: Create Groups
    group:
      name: "{{item}}"
      state: present
    loop:
      - dev
      - test

  - name: Create User
    user:
      name: "{{item.client_name}}"
      shell: /bin/bash/
      groups: test
      append: yes
      home: "/home/{{item.home_dir}}"
    with_items:
      - {client_name: test, home_dir: home_dir_test}

  - name: Installing package
    yum:
      name:
        - vim
        - mc
      state: latest
```

Модуль 13. Системы контроля версий. Распределённая система управления версиями Git

Задание: Установка git на ВМ, создать репозиторий в предварительно подготовленном сервере Git с пустым файлом Readme.md, клонировать репозитория на локаль, сделать commit с Readme.md в новой ветке с последующим merge branch в удаленном репозиторий в master.

Уровень: 3

Продолжительность выполнения: 20 мин.

Установка Git на сервер

```
$ sudo apt update -y
$ sudo apt install git -y
$ git -version
```

Конфигурация Git на сервере

Создаем пользователя git (пароль usergit)

```
$ sudo adduser git
```

Логинимся под пользователем git

```
$ sudo su - git
```

Предварительные настройки Git

```
$ git config --global user.email "git@git.git"
$ git config --global user.name "Git User"
```

Инициализируем новый репозиторий

```
$ git init ~/tempgitsrv.git
```

Добавляем пустой файл

```
$ cd ~/tempgitsrv.git
$ touch Readme.md
```

Коммитим изменения

```
$ git add Readme.md
$ git commit -m "empty Readme.md"
[master (root-commit) b0186e8] empty Readme.md
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Readme.md
```

Включите хук после обновления, скопировав файла `.sample` следующим образом:

```
# cd tempgitsrv.git/hooks/
```

```
# cp post-update.sample post-update
```

Настраиваем NAT для сервера Git

General VNF Device **NAT** DHCP IP Reservations Gateway Logs



☐	ID	Local IP	Local Port	Public IP	Public port	Protocol
☐	1	192.168.3.2	9418	176.118.164.152	9418	tcp
☐	2	192.168.3.2	22	176.118.164.152	2224	tcp

**На локальном компьютере должен быть установлен и настроен Git.
Клонируем Git локально**

```
$ mkdir ~/tempgit
$ cd ~/tempgit/
$ git clone git+ssh://git@176.118.164.152:2224/~/tempgitsrv.git
tempgitsrv.git
Cloning into 'tempgitsrv.git'...
git@176.118.164.152's password:
warning: You appear to have cloned an empty repository.
```

Создаем новую ветвь

```
$ cd tempgitsrv.git/
$ git checkout -b readme
Switched to a new branch 'readme'
```

Создаем новый файл и отправляем изменения в удаленный репозиторий

```
$ echo '# Readme' > Readme.md
$ git add Readme.md
$ git commit -m "Readme.md"
[readme 56eaae3] Readme.md
1 file changed, 1 insertion(+)
$ git push --set-upstream origin readme
git@176.118.164.152's password:
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 254 bytes | 254.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To git+ssh://176.118.164.152:2224/~/tempgitsrv.git
 * [new branch]      readme -> readme
branch 'readme' set up to track 'origin/readme'.
```

Слияние двух веток на сервере Git

На сервере, где установлен Git выводим список всех веток:

```
$ git branch -vv
* master b0186e8 empty Readme.md
```

```
readme 56eaae3 Readme.md
```

Выводим на экран содержимое файла Readme.md

```
$ cat Readme.md
```

Производим слияние ветки master и readmi

```
$ git merge readme
Updating b0186e8..56eaae3
Fast-forward
  Readme.md | 1 +
  1 file changed, 1 insertion(+)
```

Выводим на экран содержимое файла Readme.md

```
$ cat Readme.md
# Readme
```

Модуль 14. Жизненный цикл ПО

Задание: Создание ВМ, установка на нём Jenkins, развёртывание с помощью Jenkins

<https://github.com/AliyunContainerService/redis-cluster> на ВМ

Уровень: 5

Продолжительность выполнения: 40мин.

Установка Jenkins

Официальная инструкция <https://www.jenkins.io/doc/book/installing/linux/>

В Debian и дистрибутивах на основе Debian, таких как Ubuntu, вы можете установить Jenkins через apt.

Выпуск с долгосрочной поддержкой

Выпуск [LTS \(долгосрочная поддержка\)](#) выбирается каждые 12 недель из потока обычных выпусков в качестве стабильного выпуска на этот период времени. Его можно установить из [debian-stable репозитория apt](#).

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins
```

Еженедельный выпуск

Еженедельно выпускается новая версия, в которой пользователи и разработчики плагинов получают исправления ошибок и новые функции. Его можно установить из [debian репозитория apt](#).

```
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins
```

Начиная с Jenkins 2.335 и Jenkins 2.332.1, пакет настроен с использованием, а `systemd` не более старой версии System V `init`. См. [руководство сообщества DigitalOcean systemd](#), чтобы лучше понять преимущества `systemd` и `systemctl` команду.

Установка пакета:

- Настройте Jenkins как демон, запускаемый при запуске. Выполните `systemctl cat jenkins` для просмотра подробностей.
- Создайте пользователя `jenkins` для запуска этой службы.
- Прямой вывод журнала консоли в `systemd-journald`. Запустите `journalctl -u jenkins.service`, если вы устраняете неполадки Jenkins.
- Заполните `/lib/systemd/system/jenkins.service` параметры конфигурации для запуска, например, `JENKINS_HOME`
- Настройте Jenkins для прослушивания порта 8080. Получите доступ к этому порту в браузере, чтобы начать настройку.



Если Jenkins не запускается из-за того, что порт используется, запустите `systemctl edit jenkins` добавьте следующее:

```
[Service]
Environment="JENKINS_PORT=8081"
```

Здесь был выбран «8081», но вы можете указать другой доступный порт.

Установка Java

Для работы Jenkins требуется Java, но некоторые дистрибутивы не включают ее по умолчанию, а [некоторые версии Java несовместимы](#) с Jenkins.

Существует несколько реализаций Java, которые вы можете использовать. [OpenJDK](#) — самый популярный на данный момент, мы будем использовать его в этом руководстве. Обновите репозитории Debian apt, установите OpenJDK 11 и проверьте установку с помощью команд:

```
$ sudo apt update
$ sudo apt install openjdk-11-jre
$ java -version
openjdk version "11.0.12" 2021-07-20
OpenJDK Runtime Environment (build 11.0.12+7-post-Debian-2)
OpenJDK 64-Bit Server VM (build 11.0.12+7-post-Debian-2, mixed mode, sharing)
```

Запуск Jenkins

Вы можете включить службу Jenkins для запуска при загрузке с помощью команды:

```
sudo systemctl enable jenkins
```

Вы можете запустить службу Jenkins с помощью команды:

```
sudo systemctl start jenkins
```

Вы можете проверить статус службы Jenkins с помощью команды:

```
sudo systemctl status jenkins
```

Если все настроено правильно, вы должны увидеть такой вывод:

```
Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; vendor
preset: enabled)
Active: active (running) since Tue 2018-11-13 16:19:01 +03; 4min 57s
ago
```



Если у вас установлен брандмауэр, вы должны добавить Jenkins в качестве исключения. Вы должны изменить `YOURPORT` в приведенном ниже сценарии порт, который хотите использовать. Портвейн 8080 самый обычный.

```
YOURPORT=8080
PERM="--permanent"
SERV="$PERM --service=jenkins"

firewall-cmd $PERM --new-service=jenkins
firewall-cmd $SERV --set-short="Jenkins ports"
```

```
firewall-cmd $SERV --set-description="Jenkins port exceptions"
firewall-cmd $SERV --add-port=$YOURPORT/tcp
firewall-cmd $PERM --add-service=jenkins
firewall-cmd --zone=public --add-service=http --permanent
firewall-cmd --reload
```

Настраиваем NAT для сервера Jenkins

192.168.3.2	8080	176.118.164.152	8088	tcp
-------------	------	-----------------	------	-----

Мастер настройки после установки

После загрузки, установки и запуска Jenkins с помощью одной из описанных выше процедур (кроме установки с помощью Jenkins Operator) запускается мастер постустановочной настройки.

Этот мастер установки проведет вас через несколько быстрых «разовых» шагов, чтобы разблокировать Jenkins, настроить его с помощью плагинов и создать первого пользователя-администратора, через которого вы сможете продолжить доступ к Jenkins.

Разблокировка Дженкинса

При первом доступе к новому экземпляру Jenkins вас попросят разблокировать его с помощью автоматически сгенерированного пароля.

1. Найдите `http://localhost:8088` (или любой другой порт, который вы настроили для Jenkins при его установке) и подождите, пока не появится страница **Unlock Jenkins**.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

[Continue](#)

2. Из вывода журнала консоли Jenkins скопируйте автоматически сгенерированный буквенно-цифровой пароль (между двумя наборами звездочек).

```

INFO: Pre-instantiating singletons in org.springframework.beans.factory.support.DefaultListableBeanFactory@24cf7404: defining b
eans [filter,legacy]; root of factory hierarchy
Sep 30, 2017 7:18:39 AM jenkins.install.SetupWizard init
INFO:
*****
*****
*****

Jenkins initial setup is required. An admin user has been created and a password generated.
Please use the following password to proceed to installation:

2f064d3663814887964b682940572567

This may also be found at: /var/jenkins_home/secrets/initialAdminPassword

*****
*****
*****

--> setting agent port for jnlp
--> setting agent port for jnlp... done
Sep 30, 2017 7:18:51 AM hudson.model.UpdateSite updateData
INFO: Obtained the latest update center data file for UpdateSource default
Sep 30, 2017 7:18:52 AM hudson.model.UpdateSite updateData
INFO: Obtained the latest update center data file for UpdateSource default
Sep 30, 2017 7:18:52 AM hudson.WebAppMain$3 run
INFO: Jenkins is fully up and running
Sep 30, 2017 7:18:52 AM hudson.model.DownloadService$Downloadable load
INFO: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
Sep 30, 2017 7:18:58 AM hudson.model.DownloadService$Downloadable load
INFO: Obtained the updated data file for hudson.tools.JDKInstaller
Sep 30, 2017 7:18:59 AM hudson.model.AsyncPeriodicWork$1 run
INFO: Finished Download metadata. 25,543 ms

```

Примечание:

- Команда: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword` напечатает пароль на консоли.
 - Если вы используете Jenkins в Docker, используя официальный `jenkins/jenkins` образ, вы можете использовать его `sudo docker exec ${CONTAINER_ID or CONTAINER_NAME} cat /var/jenkins_home/secrets/initialAdminPassword` для печати пароля в консоли без необходимости выполнения в контейнере.
3. На странице «**Разблокировать Jenkins**» вставьте этот пароль в поле «**Пароль администратора**» и нажмите «**Продолжить**» .

Примечания:

- Вы всегда можете получить доступ к журналу консоли Jenkins из журналов Docker ([выше](#)).
- В журнале консоли Jenkins указано место (в домашнем каталоге Jenkins), где также можно получить этот пароль. Этот пароль необходимо ввести в мастере установки на новых установках Jenkins, прежде чем вы сможете получить доступ к основному пользовательскому интерфейсу Jenkins. Этот пароль также служит паролем учетной записи администратора по умолчанию (с именем пользователя «admin»), если вы пропустите следующий шаг создания пользователя в мастере установки.

Настройка Jenkins с помощью плагинов

После [разблокировки Jenkins](#) появится страница **Customize Jenkins** . Здесь вы можете установить любое количество полезных плагинов в рамках первоначальной настройки. Щелкните один из двух показанных вариантов:

- **Установить рекомендуемые плагины** — чтобы установить рекомендуемый набор плагинов, основанный на наиболее распространенных вариантах использования.
- **Выбрать плагины для установки** — выбор набора плагинов для первоначальной установки. При первом доступе к странице выбора плагинов предлагаемые плагины выбираются по умолчанию.



Если вы не уверены, какие плагины вам нужны, выберите «**Установить предлагаемые плагины**» . Вы можете установить (или удалить) дополнительные подключаемые модули Jenkins позднее через

страницу «Управление Jenkins» > Плагины в Jenkins.
--

Мастер установки показывает процесс настройки Jenkins и установки выбранного набора подключаемых модулей Jenkins. Этот процесс может занять несколько минут.

Создание первого пользователя-администратора

Наконец, после [настройки Jenkins с помощью плагинов](#) Jenkins попросит вас создать первого пользователя-администратора.

1. Когда появится страница **«Создать первого пользователя-администратора»**, укажите сведения о вашем пользователе-администраторе в соответствующих полях и нажмите **«Сохранить и завершить»**.
2. Когда появится страница **Jenkins is ready**, нажмите **Start using Jenkins**.

Примечания:

- Эта страница может означать, что **Дженкинс почти готов!** вместо этого, и если это так, нажмите **«Перезагрузить»**.
 - Если страница не обновляется автоматически через минуту, обновите страницу вручную с помощью веб-браузера.
3. При необходимости войдите в Jenkins с учетными данными только что созданного пользователя, и вы готовы начать использовать Jenkins!

Модуль 15. Практические навыки работы с Docker

Задание: Создание dockerfile на базе образа centos 7.9, в него установить PostgreSQL версии не ниже 12, добавить пользователя в БД, создать в БД 2 таблицы и наполнить их на 10 значений. Скрипты и наполнение можно взять из модуля 6. Собрать полученный docker image через docker build и запустить. Подключиться с хостовой машины к инсталлированному PostgreSQL утилитой psql.

Уровень: 8

Продолжительность выполнения: 40 мин.

Dockerfile:

```
#####
#
# docker build --rm -t local/c7-9-pg .
#
# docker run -p 5432:5432 -d local/c7-9-pg
#
# docker run --name pgadmin -p 5051:80 \
# -e "PGADMIN_DEFAULT_EMAIL=user@pgadmin.com" \
# -e "PGADMIN_DEFAULT_PASSWORD=pgadmin" -d dpape/pgadmin4
#####
FROM centos:7.9.2009
COPY ./startUpScript.sh /
COPY ./postgresql.conf /
RUN yum install -y epel-release wget \
&& yum clean all \
&& yum install -y
https://download.postgresql.org/pub/repos/yum/reporepms/EL-7-
x86_64/pgdg-redhat-repo-latest.noarch.rpm \
&& yum install -y postgresql12-server postgresql12-contrib \
&& chown root /startUpScript.sh \
&& chgrp root /startUpScript.sh \
&& chmod 777 /startUpScript.sh

EXPOSE 5432

CMD ["/bin/bash","-c","/startUpScript.sh && tail -f /dev/null"]
```

postgresql.conf

```
listen_addresses = '*'
port = 5432
logging_collector = on
log_filename = 'postgresql-%a.log'
log_truncate_on_rotation = on
log_rotation_age = 1d
log_rotation_size = 0
```

startUpScript.sh

```
#!/bin/bash
su -l postgres -c /usr/pgsql-12/bin/initdb
```

```
echo "host      all                          all          0.0.0.0/0
trust" >> /var/lib/pgsql/12/data/pg_hba.conf
cp ./postgresql.conf /var/lib/pgsql/12/data/postgresql.conf
su -l postgres -c "/usr/pgsql-12/bin/pg_ctl -D /var/lib/pgsql/12/data
-l /tmp/pg_logfile start"

set -e

psql -v ON_ERROR_STOP=1 --username "postgres" --dbname "postgres" <<-
EOSQL
    CREATE USER tester;
    CREATE DATABASE "tempdb-15";
    GRANT ALL PRIVILEGES ON DATABASE "tempdb-15" TO tester;
    CREATE TABLE snowboarder ( equip_id serial PRIMARY KEY, title
varchar (50) NOT NULL,company varchar (25) NOT NULL, size varchar (25)
check (size in ('XS', 'S', 'M', 'L', 'XL', 'XXL')));
    INSERT INTO snowboarder (title, company, size) VALUES
('snowboard', 'burton', 'XL');
    INSERT INTO snowboarder (title, company, size) VALUES ('binding',
'burton', 'XL');
    INSERT INTO snowboarder (title, company, size) VALUES ('boots',
'burton', 'XL');
EOSQL
```

Подключаемся с хостовой машины и проверяем созданную таблицу с данными:

```
psql -h localhost -U postgres
psql (12.14 (Ubuntu 12.14-0ubuntu0.20.04.1))
Type "help" for help.

postgres=# \dt
               List of relations
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 public | snowboarder    | table | postgres
(1 row)

postgres=# select * from snowboarder;
 equip_id |   title   | company | size
-----+-----+-----+-----
        1 | snowboard | burton  | XL
        2 | binding  | burton  | XL
        3 | boots    | burton  | XL
(3 rows)

postgres=# exit
```

Модуль 16. Микросервисы и микросервисная архитектура

Задание: Создать свой docker-compose file на базе v3, в котором описать развёртывание в одном контейнере PHP+ tomcat 8 и во втором пустой БД MariaDB, в которой будет замаплен локальный volume ./mysql/. Собрать и запустить, показать страницу php из браузера и доступность порта MariaDB

Уровень: 5

Продолжительность выполнения: 35 мин.

Для запуска PHP скрипта на Tomcat нужен мост PHP-JAVA.

Будем использовать <https://php-java-bridge.sourceforge.net/pjb/>.

Создаем рабочую директорию.

Создаем Dockerfile с Tomcat и PHP/Java Bridge:

```
FROM tomcat:8.5
EXPOSE 8080
RUN apt-get install php-cgi -y
ENTRYPOINT ["catalina.sh", "run"]
```

Создаем образ

```
docker build --rm -t local/tomcatphp .
```

Создаем директорию ./webapps

В директорию скачиваем вебархив

```
wget https://nav.dl.sourceforge.net/project/php-java-bridge/Binary%20package/php-java-bridge_7.2.1/JavaBridgeTemplate721.war -O ./webapps/JavaBridge.war
```

Готовим docker compose файл

```
version: '3'
services:

  #PHP Service
  app:
    image: local/tomcatphp-fpm
    container_name: app
    volumes:
      - ./webapps:/usr/local/tomcat/webapps
    ports:
      - 9980:8080

  db:
    image: mariadb
    restart: always
    ports:
      - 3306:3306
    volumes:
      - ./mysql:/var/lib/mysql
```

```
environment:
  MARIADB_ROOT_PASSWORD: example
```

Запускаем сервисы:

```
docker compose up -d
```

После того как сервисы запустятся и JavaBridge.war установится в ./webapps/ копируем в указанную директорию файл test.php

```
<?php phpinfo(); ?>
```

Проверяем доступность файла test.php

```
curl localhost:9980/JavaBridge/test.php
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"DTD/xhtml11-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"><head>
<style type="text/css">
body {background-color: #fff; color: #222; font-family: sans-serif;}
pre {margin: 0; font-family: monospace;}
a:link {color: #009; text-decoration: none; background-color: #fff;}
a:hover {text-decoration: underline;}
table {border-collapse: collapse; border: 0; width: 934px; box-shadow:
1px 2px 3px #ccc;}
.center {text-align: center;}
.center table {margin: 1em auto; text-align: left;}
.center th {text-align: center !important;}
td, th {border: 1px solid #666; font-size: 75%; vertical-align:
baseline; padding: 4px 5px;}
th {position: sticky; top: 0; background: inherit;}
h1 {font-size: 150%;}
h2 {font-size: 125%;}
.p {text-align: left;}
.e {background-color: #ccf; width: 300px; font-weight: bold;}
.h {background-color: #99c; font-weight: bold;}
.v {background-color: #ddd; max-width: 300px; overflow-x: auto; word-
wrap: break-word;}
.v i {color: #999;}
img {float: right; border: 0;}
hr {width: 934px; background-color: #ccc; border: 0; height: 1px;}
</style>
<title>PHP 8.1.2-1ubuntu2.11 - phpinfo()</title><meta name="ROBOTS"
content="NOINDEX,NOFOLLOW,NOARCHIVE" /></head>
...
```

Проверяем доступность порта MariaDB

```
sudo netstat -ltnp | grep -w ':3306'
[sudo] password for user:
tcp6      0      0 :::3306          :::*
LISTEN    58535/docker-proxy
```

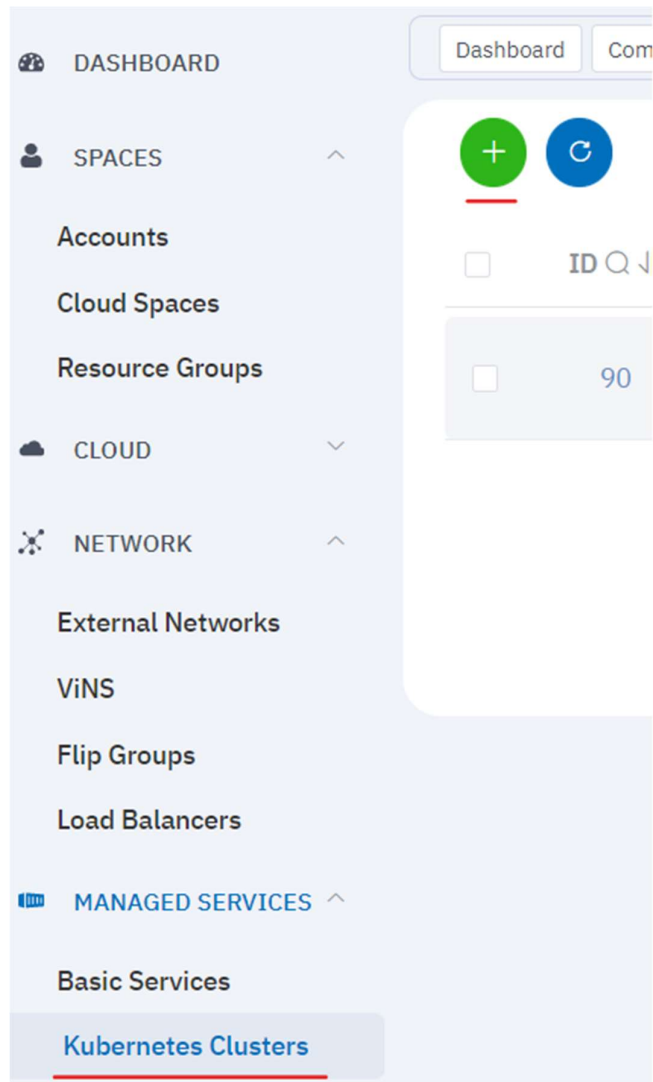
Модуль 17. Оркестровка контейнеров, кластеры Kubernetes

Задание: Развернуть k8s v 1.26 из 1 master и 1 worker node и установить на worker node RabbitMQ через Helm с публикацией через ingress

Уровень: 5

Продолжительность выполнения: 40мин.

Создание кластера Kubernetes



×

Create a new K8s

1

2

3

4

General

Master Node

Worker Node

Network

* Name

k8s-test

* Account

Universiade_education

▼

* Resource Group

Test

▼

* K8ci Instance

k8s-1.24-u22-gfsc

▼

* Worker Group Name

wg-test

Description

Next Step

×

Create a new K8s

✓

General

2

Master Node

3

Worker Node

4

Network

Master Nums

—

1

+

* CPU

—

2

+

* Memory in MB

—

2048

+

* Vdisk in GB

—

10

+

* SEP and Pool

☒ Automatically

☐ Manually

Previous step

Next Step

Create a new K8s

✓

✓

3

4

General

Master Node

Worker Node

Network

Worker Nums

—

1

+

* CPU

—

1

+

* Memory in MB

—

1024

+

* Vdisk in GB

—

10

+

* SEP and Pool

☒ Automatically

☐ Manually

Labels

key=value then press enter

▼

Taints

key=value then press enter

▼

Annotations

key=value then press enter

▼

Previous step

Next Step

Create a new K8s

✓

✓

✓

4

General

Master Node

Worker Node

Network

External Network

Select External Network

▼

Previous step

Confirm Creating

Установка kubectr

<https://kubernetes.io/ru/docs/tasks/tools/install-kubectl/>

Обновите apt индекс пакетов и установите пакеты, необходимые для использования apt репозитория Kubernetes:

```
$ sudo apt-get update
$ sudo apt-get install -y ca-certificates curl
```

Если вы используете Debian 9 (stretch) или более раннюю версию, вам также потребуется установить apt-transport-https:

```
$ sudo apt-get install -y apt-transport-https
```

Загрузите открытый ключ подписи Google Cloud:

```
$ sudo curl -fsSLo /etc/apt/keyrings/kubernetes-archive-keyring.gpg
https://packages.cloud.google.com/apt/doc/apt-key.gpg
```

Добавьте репозиторий Kubernetes apt:

```
$ echo "deb [signed-by=/etc/apt/keyrings/kubernetes-archive-
keyring.gpg] https://apt.kubernetes.io/ kubernetes-xenial main" | sudo
tee /etc/apt/sources.list.d/kubernetes.list
```

Обновите apt индекс пакета с помощью нового репозитория и установите kubectl:

```
$ sudo apt-get update
$ sudo apt-get install -y kubectl
```



Примечание. В выпусках старше Debian 12 и Ubuntu 22.04 /etc/apt/keyrings не существует по умолчанию. Вы можете создать этот каталог, если вам нужно, сделав его доступным для чтения всем, но доступным для записи только администраторам.

Подключение к созданному кластеру

Сохраняем файл конфигурации

Dashboard / Managed Services / K8S Service

Dashboard Compute 10810 Resource Groups Resource Group 1464 VINS 908 Kubernetes Clusters Resource Group 1677 K8S

General Basic Service Master Workers

Parameter	Value
Kuber Config	
ID	332
Name	k8s-test
Account ID	97
RG ID	1677
Status	ENABLED
Tech.Status	STARTED

В корень рабочей директории копируем файл конфигурации с именем config.kuber
Выводим список нод:

```
$ kubectl --kubeconfig config.kuber get nodes
```

Установка пакетного менеджера Helm

<https://helm.sh/docs/intro/install/>

Из Apt (Debian/Ubuntu)

Члены сообщества Helm предоставили пакет Helm для Apt. Этот пакет, как правило, актуален.

```
$ curl https://baltocdn.com/helm/signing.asc | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
$ sudo apt-get install apt-transport-https --yes
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/helm.gpg] https://baltocdn.com/helm/stable/debian/ all main" | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list
$ sudo apt-get update
$ sudo apt-get install helm
```

Подготовка StorageClass, PersistentVolume и PersistentVolumeClaim

```
$ cat > storageClass-rabbitmq.yaml << EOF
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: rabbitmq-local-storage
  namespace: ns-rabbitmq
provisioner: kubernetes.io/no-provisioner
```

```
volumeBindingMode: WaitForFirstConsumer
EOF

$ kubectl create -f storageClass-rabbitmq.yaml
```

На ворк ноде (**node1** – нужно указать имя ноды из конфигурационной панели кластера **k8s Basis**) нужно создать директорию для persistentVolume:

```
# on the node, where the POD will be located (node1 in our case):
DIRNAME="vol1"
mkdir -p /mnt/disk/$DIRNAME
chcon -Rt svirt_sandbox_file_t /mnt/disk/$DIRNAME
chmod 777 /mnt/disk/$DIRNAME
```

```
$ cat > persistentVolume-rabbitmq.yaml << EOF
apiVersion: v1
kind: PersistentVolume
metadata:
  name: rabbitmq-local-pv
  namespace: ns-rabbitmq
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: my-local-storage
  local:
    path: /mnt/disk/vol1
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - node1
EOF

$ kubectl create -f persistentVolume-rabbitmq.yaml
```

```
$ cat > persistentVolumeClaim-rabbitmq.yaml << EOF
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: rabbitmq-claim
  namespace: ns-rabbitmq
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: my-local-storage
```

```
resources:
  requests:
    storage: 5Gi
EOF

$ kubectl create -f persistentVolumeClaim-rabbitmq.yaml
```

Установка RabbitMQ через Helm в Kubernetes

Для установки RabbitMQ воспользуемся Helm-чартом bitnami/rabbitmq. Достоинство чартов в том, что не нужно устанавливать по отдельности все необходимые сервису ресурсы: можно установить их одновременно в рамках общего релиза. А при изменениях в любом из ресурсов можно вынести новый релиз, в котором все обновления будут собраны воедино.

Создадим папку helm, добавим в нее репозиторий bitnami и найдем нужный нам Helm Chart bitnami/rabbitmq:

```
$ mkdir helm
$ cd helm
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm search repo bitnami | grep rabbitmq
bitnami/rabbitmq 11.11.1
3.11.10 RabbitMQ is an open source general-purpose mess...
$ bitnami/rabbitmq-cluster-operator 3.2.7
2.2.0 The RabbitMQ Cluster Kubernetes Operator automa...
```

Копируем его имя, загружаем и распаковываем:

```
$ helm pull bitnami/rabbitmq
$ tar -zxvf rabbitmq-11.11.1.tgz
```

Переходим в папку rabbitmq/templates. Здесь находятся все ресурсы, которые нужно будет создать в Kubernetes для корректной работы RabbitMQ: конфигурация, Ingress, сертификаты, сетевые политики, сервисные аккаунты, секреты, правила Prometheus и так далее. И Helm позволяет это сделать единой командой, без установки каждого файла по отдельности:

```
-rw-r--r-- 1 vozerov staff 6328 Nov 26 14:02 NOTES.txt
-rw-r--r-- 1 vozerov staff 7108 Nov 26 14:02 _helpers.tpl
-rw-r--r-- 1 vozerov staff 719 Nov 26 14:02 certs.yaml
-rw-r--r-- 1 vozerov staff 525 Nov 26 14:02 configuration.yaml
-rw-r--r-- 1 vozerov staff 1508 Nov 26 14:02 ingress.yaml
-rw-r--r-- 1 vozerov staff 1248 Nov 26 14:02 networkpolicy.yaml
-rw-r--r-- 1 vozerov staff 548 Nov 26 14:02 pdb.yaml
-rw-r--r-- 1 vozerov staff 879 Nov 26 14:02 prometheusrule.yaml
-rw-r--r-- 1 vozerov staff 417 Nov 26 14:02 role.yaml
-rw-r--r-- 1 vozerov staff 528 Nov 26 14:02 rolebinding.yaml
-rw-r--r-- 1 vozerov staff 1174 Nov 26 14:02 secrets.yaml
-rw-r--r-- 1 vozerov staff 317 Nov 26 14:02 serviceaccount.yaml
-rw-r--r-- 1 vozerov staff 1463 Nov 26 14:02 servicemonitor.yaml
-rw-r--r-- 1 vozerov staff 15192 Nov 26 14:02 statefulset.yaml
-rw-r--r-- 1 vozerov staff 897 Nov 26 14:02 svc-headless.yaml
-rw-r--r-- 1 vozerov staff 3636 Nov 26 14:02 svc.yaml
-rw-r--r-- 1 vozerov staff 361 Nov 26 14:02 tls-secrets.yaml
```

Возвращаемся в родительскую папку helm, чтобы посмотреть возможность настройки файла values.yaml. Скопируем содержимое rabbitmq/values.yaml в наш собственный файл values.dev.yaml и откроем его для редактирования:

```
$ cp rabbitmq/values.yaml ./values.dev.yaml
$ vi values.dev.yaml
```

Так поступать рекомендуется всегда, так как настройки для разных сред будут отличаться. В данном файле содержится очень много параметров, которые можно настраивать под нужды своего проекта: режим debug, плагины RabbitMQ для подключения, необходимость включения TLS и memoryHighWatermark, аутентификация через LDAP, количество реплик, nodeSelector для создания RabbitMQ на нодах с определенной меткой, требования к CPU и памяти и многое другое.

Нас в первую очередь интересуют настройки Ingress. Находим секцию ingress, устанавливаем в enabled значение true/

```
## Configure the ingress resource that allows you to access the
## RabbitMQ installation. Set up the URL
## ref: https://kubernetes.io/docs/user-guide/ingress/
##
ingress:
  ## @param ingress.enabled Enable ingress resource for Management console
  ##
  enabled: true

  ## @param ingress.path Path for the default host. You may need to set this to '/' in order to use this with ALB ingress controllers.
  ##
  path: /
```

Указываем созданный persistentVolumeClaim

```
## @param persistence.existingClaim Provide an existing PersistentVolumeClaims
## The value is evaluated as a template
## So, for example, the name can depend on .Release or .Chart
##
existingClaim: "rabbitmq-claim"
```

Далее переходим непосредственно к разворачиванию RabbitMQ. Создаем новый namespace ns-rabbitmq и применяем к нему созданный файл values.dev.yaml:

```
$ kubectl --kubeconfig ../config.kuber create ns ns-rabbitmq
$ helm --kubeconfig ../config.kuber install -n ns-rabbitmq rabbitmq -f
values.dev.yaml ./rabbitmq/
```

```
WARNING: Kubernetes configuration file is group-readable. This is insecure. Location: ../config.kuber
WARNING: Kubernetes configuration file is world-readable. This is insecure. Location: ../config.kuber
NAME: rabbitmq
LAST DEPLOYED: Tue Apr 11 14:03:08 2023
NAMESPACE: ns-rabbitmq
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: rabbitmq
CHART VERSION: 11.11.1
APP VERSION: 3.11.10** Please be patient while the chart is being deployed **

Credentials:
  echo "Username      : user"
  echo "Password      : $(kubectl get secret --namespace ns-rabbitmq rabbitmq -o jsonpath="{.data.rabbitmq-password}" | base64 -d)"
  echo "Erlang Cookie : $(kubectl get secret --namespace ns-rabbitmq rabbitmq -o jsonpath="{.data.rabbitmq-erlang-cookie}" | base64 -d)"

Note that the credentials are saved in persistent volume claims and will not be changed upon upgrade or reinstallation unless the persistent volume claim has been deleted. If this is not the first installation of this chart, the credentials may not be valid.
This is applicable when no passwords are set and therefore the random password is autogenerated. In case of using a fixed password, you should specify it when upgrading.
More information about the credentials may be found at https://docs.bitnami.com/general/how-to/troubleshoot-helm-chart-issues/#credential-errors-while-upgrading-chart-releases.

RabbitMQ can be accessed within the cluster on port 5672 at rabbitmq.ns-rabbitmq.svc.cluster.local

To access for outside the cluster, perform the following steps:

To Access the RabbitMQ AMQP port:
1. Create a port-forward to the AMQP port:

  kubectl port-forward --namespace ns-rabbitmq svc/rabbitmq 5672:5672 &
  echo "URL : amqp://127.0.0.1:5672/"

2. Access RabbitMQ using using the obtained URL.

To Access the RabbitMQ Management interface:
1. Get the RabbitMQ Management URL and associate its hostname to your cluster external IP:

  export CLUSTER_IP=$(minikube ip) # On Minikube, use: 'kubectl cluster-info' on others K8s clusters
  echo "RabbitMQ Management: http://rabbitmq.local/"
  echo "$CLUSTER_IP rabbitmq.local" | sudo tee -a /etc/hosts

2. Open a browser and access RabbitMQ Management using the obtained URL.
```

Проверяем статус запущенного пода

```
$ kubectl --kubeconfig ../config.kuber get pods -n ns-rabbitmq
```

NAME	READY	STATUS	RESTARTS	AGE
rabbitmq-0	1/1	Running	0	53m

Посмотрим Persistent Volumes, выделенные RabbitMQ

```
$ kubectl --kubeconfig ../config.kuber get pvc -n ns-rabbitmq
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
rabbitmq-claim	Bound	rabbitmq-local-pv	10Gi	RWO
my-local-storage	154m			

Модуль 18. Kubernetes конфигурация развертывания

Задание: Установить в k8s cluster из п.17, Prometheus+Grafana.

Уровень: 8

Продолжительность выполнения: 40 мин.

Установка HELM на Linux

Helm — это установщик пакетов для Kubernetes. Обычно его устанавливают локально на ОС, с которой управляется Kubernetes. Helm нам нужен для быстрой установки пакетов Kube-Prometheus-Stack в Kubernetes. Чтобы осталась возможность обновлять helm, рекомендуем сделать установку через apt:

```
$ curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -
$ sudo apt-get install apt-transport-https -yes
$ echo "deb https://baltocdn.com/helm/stable/debian/ all main" | sudo
tee /etc/apt/sources.list.d/helm-stable-debian.list
$ sudo apt-get update
$ sudo apt-get install helm
```

Поиск диаграммы Prometheus Helm Chart

Вы можете искать Prometheus Helm из командной строки следующим образом:

```
$ helm search hub prometheus
```

Команда отображает следующие диаграммы Helm:

URL	CHART VERSION	APP VERSION	DESCRIPTION
https://artifacthub.io/packages/helm/prometheus...	19.0.0	v2.40.5	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/prometheus...	13.0.0	2.22.1	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/saurabhe-p...	0.2.0	1.1	This is a helm chart for Prometheus Setup.
https://artifacthub.io/packages/helm/stakater/p...	1.0.32		Prometheus chart that runs on Kubernetes
https://artifacthub.io/packages/helm/cloudposse...	0.2.1		Prometheus instance created by the CoreOS Prome...
https://artifacthub.io/packages/helm/pascaliske...	1.3.4	v2.40.5	A Helm chart for Prometheus
https://artifacthub.io/packages/helm/banzaiclou...	7.3.4-thenos.4	2.4.3	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/azureorke...	14.8.0	2.26.0	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/openstack-...	0.1.7	v2.25.0	OpenStack-Helm Prometheus
https://artifacthub.io/packages/helm/edu/promet...	11.6.0	2.19.0	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/k8s-edu/pr...	15.8.5	2.34.0	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/wenerme/pr...	19.0.0	v2.40.5	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/wener/pron...	19.0.0	v2.40.5	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/cloudave/pr...	6.2.1	2.2.1	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/osc/promet...	0.15.2	v2.35.0	OSC Prometheus deployment
https://artifacthub.io/packages/helm/rgnu/prom...	0.1.1	2.4.3	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/tnh/promet...	11.6.0	2.19.0	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/cloudnativ...	8.11.4	2.9.2	Prometheus is a monitoring system and time seri...
https://artifacthub.io/packages/helm/prometheus...	3.4.2	v0.10.0	A Helm chart for k8s prometheus adapter
https://artifacthub.io/packages/helm/mesosphere...	12.11.13	0.44.0	prometheus-operator collects Kubernetes manifes...
https://artifacthub.io/packages/helm/prometheus...	2.0.2	v1.5.1	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/prometheus...	2.10.0	v0.8.2	A Helm chart for k8s prometheus adapter
https://artifacthub.io/packages/helm/arldka/pro...	2.0.1	0.60.1	Stripped down version of prometheus-operator to...
https://artifacthub.io/packages/helm/prometheus...	1.5.0	3.0	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/wener/kube...	8.2.2	0.61.1	Prometheus Operator provides easy monitoring de...
https://artifacthub.io/packages/helm/bitnami-ak...	8.1.11	0.60.1	Prometheus Operator provides easy monitoring de...
https://artifacthub.io/packages/helm/bitnami/ku...	8.2.2	0.61.1	Prometheus Operator provides easy monitoring de...
https://artifacthub.io/packages/helm/bitnami-ak...	0.29.3	0.41.0	The Prometheus Operator for Kubernetes provides...
https://artifacthub.io/packages/helm/choerodon/...	9.3.1	0.38.1	Provides easy monitoring definitions for kubern...
https://artifacthub.io/packages/helm/cloudnativ...	1.0.2	v0.5.0	A Helm chart for k8s prometheus adapter
https://artifacthub.io/packages/helm/verik-char...	1.0.3	1.0.0	Chart for configure prometheus federation
https://artifacthub.io/packages/helm/portefaix-...	1.1.1	2.31.1	A Helm chart for Prometheus Mixin
https://artifacthub.io/packages/helm/cloudnativ...	6.4.0	0.31.1	Provides easy monitoring definitions for kubern...
https://artifacthub.io/packages/helm/camptocamp...	5.15.1	0.31.1	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/stakater/p...	1.0.12		prometheus-operator chart that runs on Kubernetes
https://artifacthub.io/packages/helm/wenerme/pr...	2.0.1	v1.5.1	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/wener/pron...	2.0.1	v1.5.1	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/giantswarm...	1.1.0	1.4.2	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/cloudnativ...	0.4.0	0.8.0	A Helm chart for prometheus pushgateway
https://artifacthub.io/packages/helm/t3n/stackd...	0.1.0	0.4.3	A variant of the Prometheus server that can sen...
https://artifacthub.io/packages/helm/geek-cookb...	6.4.2	0.0.1	Prometheus Exporter for the official uptimebo...

Мы будем использовать официальную диаграмму Prometheus Helm от Prometheus. Давайте добавим репозиторий prometheus-community для нашей диаграммы Prometheus Helm следующим образом:

```
$ helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
```

Чтобы обновить добавленный репозиторий, выполните следующую команду:

```
$ helm repo update
```

Это дает следующий результат:

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
...Successfully got an update from the "my-repo" chart repository
Update Complete. ☺Happy Helming!☺
```

Установка Prometheus Helm Chart

Установка Prometheus Helm Chart загрузит или развернет Prometheus Helm Chart в кластер Minikube Kubernetes. Чтобы установить Prometheus Helm Chart, используйте эту команду «helm install»:

```
$ helm install prometheus prometheus-community/prometheus
```

Установка Prometheus Helm Chart дает следующий результат:

```
NAME: prometheus
LAST DEPLOYED: Mon Dec 5 16:29:21 2022
NAMESPACE: default
STATUS: deployed
REVISION: 1
NOTES:
The Prometheus server can be accessed via port 80 on the following DNS name from within your cluster:
prometheus-server.default.svc.cluster.local

Get the Prometheus server URL by running these commands in the same shell:
export POD_NAME=$(kubectl get pods --namespace default -l "app=prometheus,component=server" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace default port-forward $POD_NAME 9090

The Prometheus alertmanager can be accessed via port 9093 on the following DNS name from within your cluster:
prometheus-%s(<nil>).default.svc.cluster.local

Get the Alertmanager URL by running these commands in the same shell:
export POD_NAME=$(kubectl get pods --namespace default -l "app=prometheus,component=" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace default port-forward $POD_NAME 9093

##### WARNING: Pod Security Policy has been disabled by default since #####
##### it deprecated after k8s 1.25+, use #####
##### (index .Values "prometheus-node-exporter" "rbac" #####
##### "pspEnabled") with (index .Values #####
##### "prometheus-node-exporter" "rbac" "pspAnnotations") #####
##### in case you still need it. #####
#####

The Prometheus PushGateway can be accessed via port 9091 on the following DNS name from within your cluster:
prometheus-prometheus-pushgateway.default.svc.cluster.local

Get the PushGateway URL by running these commands in the same shell:
export POD_NAME=$(kubectl get pods --namespace default -l "app=prometheus-pushgateway,component=pushgateway" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace default port-forward $POD_NAME 9091

For more information on running Prometheus, visit:
https://prometheus.io/
```

Когда вы запускаете команду «helm install»:

- Kubernetes считывает файлы конфигурации YAML в диаграмме Helm.
- Затем он отправляет YAML-файлы конфигурации Deployments, Services, ConfigMaps и Secrets на сервер API Kubernetes.
- Kubernetes берет эти файлы и создает приложение Prometheus внутри кластера Kubernetes.
- Он установит сервер Prometheus, диспетчер предупреждений и компоненты мониторинга.

Чтобы получить все развернутые ресурсы Kubernetes для приложения Prometheus Kubernetes, выполните эту команду kubectl:

```
$ kubectl get all
```

Команда отобразит все созданные объекты/компоненты Kubernetes в кластере Kubernetes, как показано ниже:

```

NAME                                READY   STATUS    RESTARTS   AGE
pod/prometheus-alertmanager-0       1/1     Running   0           16m
pod/prometheus-kube-state-metrics-84bb75bf7d-xr51v  1/1     Running   0           16m
pod/prometheus-prometheus-node-exporter-vn8b2      1/1     Running   0           16m
pod/prometheus-prometheus-pushgateway-7545cc8db-svz5f  1/1     Running   0           16m
pod/prometheus-server-7647d5bd9b-dct2n            2/2     Running   0           16m

```

```

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/kubernetes                  ClusterIP      10.96.0.1        <none>            443/TCP          30m
service/prometheus-alertmanager     ClusterIP      10.104.50.77     <none>            9093/TCP          16m
service/prometheus-alertmanager-headless ClusterIP      None             <none>            9093/TCP          16m
service/prometheus-kube-state-metrics ClusterIP      10.109.139.238   <none>            8080/TCP          16m
service/prometheus-prometheus-node-exporter ClusterIP      10.100.135.238   <none>            9100/TCP          16m
service/prometheus-prometheus-pushgateway ClusterIP      10.103.224.84    <none>            9091/TCP          16m
service/prometheus-server           ClusterIP      10.104.207.27    <none>            80/TCP           16m

```

```

NAME                                DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
daemonset.apps/prometheus-prometheus-node-exporter 1          1         1         1             1           <none>          16m

```

```

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/prometheus-kube-state-metrics 1/1      1             1           16m
deployment.apps/prometheus-prometheus-pushgateway 1/1      1             1           16m
deployment.apps/prometheus-server             1/1      1             1           16m

```

```

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/prometheus-kube-state-metrics-84bb75bf7d 1          1         1           16m
replicaset.apps/prometheus-prometheus-pushgateway-7545cc8db 1          1         1           16m
replicaset.apps/prometheus-server-7647d5bd9b              1          1         1           16m

```

Мы создали модули, развертывания Kubernetes и сервисы. Helm упрощает создание всех этих объектов Kubernetes с помощью одной команды.

Получить все созданные сервисы Kubernetes

Чтобы получить все созданные сервисы Kubernetes, выполните следующую команду:

```
kubectl get service
```

Он отображает следующие службы Kubernetes:

```

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kubernetes                          ClusterIP      10.96.0.1        <none>            443/TCP          72m
prometheus-alertmanager             ClusterIP      10.104.50.77     <none>            9093/TCP          59m
prometheus-alertmanager-headless     ClusterIP      None             <none>            9093/TCP          59m
prometheus-kube-state-metrics        ClusterIP      10.109.139.238   <none>            8080/TCP          59m
prometheus-prometheus-node-exporter   ClusterIP      10.100.135.238   <none>            9100/TCP          58m
prometheus-prometheus-pushgateway     ClusterIP      10.103.224.84    <none>            9091/TCP          58m
prometheus-server                   ClusterIP      10.104.207.27    <none>            80/TCP           59m

```

Prometheus Helm Chart создал шесть сервисов Kubernetes. Мы будем получать доступ к приложению Prometheus с помощью службы Kubernetes prometheus-server.

Служба prometheus-server имеет тип службы Kubernetes ClusterIP. Он позволяет получить доступ к приложению Prometheus только внутри кластера Kubernetes.

Доступ к приложению Prometheus Kubernetes

Чтобы получить доступ к приложению Prometheus Kubernetes вне кластера Kubernetes, выполните следующую команду:

```
$ kubectl expose service prometheus-server --type=NodePort --target-port=9090 --name=prometheus-server-ext
```

Команда преобразует службу Kubernetes prometheus-server из типа ClusterIP в тип NodePort.

Это позволит нам получить доступ к приложению Prometheus за пределами кластера Kubernetes через порт «9090».

Команда выводит следующее:

```
service/prometheus-server-ext exposed
```

Чтобы проверить недавно созданную службу «prometheus-server-ext» Kubernetes, выполните следующую команду:

```
$ kubectl get service
```

Команда выводит следующее:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	72m
prometheus-alertmanager	ClusterIP	10.104.50.77	<none>	9093/TCP	59m
prometheus-alertmanager-headless	ClusterIP	None	<none>	9093/TCP	59m
prometheus-kube-state-metrics	ClusterIP	10.109.139.238	<none>	8080/TCP	59m
prometheus-prometheus-node-exporter	ClusterIP	10.100.135.238	<none>	9100/TCP	58m
prometheus-prometheus-pushgateway	ClusterIP	10.103.224.84	<none>	9091/TCP	58m
prometheus-server	ClusterIP	10.104.207.27	<none>	80/TCP	59m
prometheus-server-ext	NodePort	10.101.163.153	<none>	80:31011/TCP	2m27s

Вы должны иметь доступ к службе Kubernetes prometheus-server-ext за пределами кластера Kubernetes, используя URL-адрес.

Чтобы сгенерировать URL-адрес для доступа к этой службе, выполните следующую команду:

```
$ kubectl service prometheus-server-ext
```

Команда выводит следующий URL:

```
! Executing "docker container inspect minikube --format={{.State.Status}}" took an unusually long time: 3.2945493s
! Restarting the docker service may improve performance.

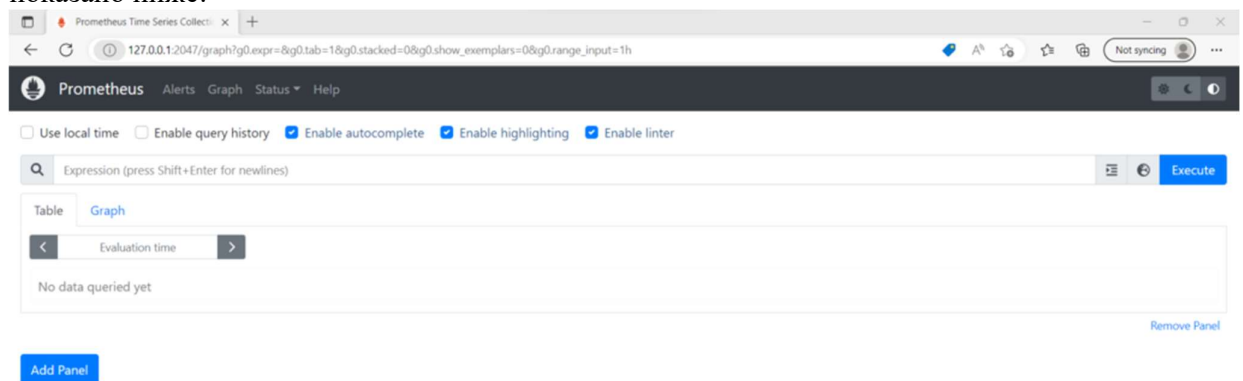
NAMESPACE      NAME                TARGET PORT      URL
-----
default        prometheus-server-ext 80              http://192.168.49.2:31011

! Starting tunnel for service prometheus-server-ext.

NAMESPACE      NAME                TARGET PORT      URL
-----
default        prometheus-server-ext 80              http://127.0.0.1:2047

! Opening service default/prometheus-server-ext in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Чтобы получить доступ к приложению Prometheus, введите URL-адрес в браузере, как показано ниже:



На изображении выше мы установили приложение Prometheus внутри кластера Kubernetes. Это веб-интерфейс Prometheus.

Поисках диаграммы Grafana Helm Chart

Мы будем использовать официальный Grafana Helm от Grafana. Давайте добавим репозиторий grafana для нашей диаграммы Grafana Helm Chart следующим образом:

```
$ helm repo add grafana https://grafana.github.io/helm-charts
```

Чтобы обновить добавленный репозиторий Grafana, выполните следующую команду:

```
$ helm repo update
```

Это дает следующий результат:

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "grafana" chart repository
...Successfully got an update from the "prometheus-community" chart repository
...Successfully got an update from the "my-repo" chart repository
Update Complete. Happy Helming!
```

Установка диаграммы Grafana Helm

Установка Grafana Helm Chart загрузит или развернет Grafana Helm Chart в кластер Kubernetes. Чтобы установить Grafana Helm Chart, используйте эту команду «helm install»:

```
$ helm install grafana grafana/grafana
```

Установка Grafana Helm Chart дает следующий результат:

```
1205 18:18:58.078119 34736 warnings.go:70] policy/v1beta1 PodSecurityPolicy is deprecated in v1.21+, unavailable in v1.25+
W1205 18:19:06.912566 34736 warnings.go:70] policy/v1beta1 PodSecurityPolicy is deprecated in v1.21+, unavailable in v1.25+
NAME: grafana
LAST DEPLOYED: Mon Dec 5 18:18:56 2022
NAMESPACE: default
STATUS: deployed
REVISION: 1
NOTES:
1. Get your 'admin' user password by running:

    kubectl get secret --namespace default grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo

2. The Grafana server can be accessed via port 80 on the following DNS name from within your cluster:

    grafana.default.svc.cluster.local

    Get the Grafana URL to visit by running these commands in the same shell:
    export POD_NAME=$(kubectl get pods --namespace default -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=grafana" -o jsonpath="{.items[0].metadata.name}")
    kubectl --namespace default port-forward $POD_NAME 3000

3. Login with the password from step 1 and the username: admin

#####
##### WARNING: Persistence is disabled!!! You will lose your data when #####
##### the Grafana pod is terminated. #####
#####
```

Когда вы запускаете команду «helm install»:

- Kubernetes считывает файлы конфигурации YAML в диаграмме Helm.
- Затем он отправляет YAML-файлы конфигурации Deployments, Services, ConfigMaps и Secrets на сервер API Kubernetes.
- Kubernetes берет эти файлы и создает приложение Grafana внутри кластера Kubernetes.

Список установленных карт Helm

Чтобы получить список двух установленных диаграмм Helm, выполните следующую команду:

```
$ helm list
```

Команда дает следующий вывод:

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
grafana	default	1	2022-12-05 18:18:56.3872495 +0300 EAT	deployed	grafana-6.45.0	9.3.0
prometheus	default	1	2022-12-05 16:29:21.2691582 +0300 EAT	deployed	prometheus-19.0.0	v2.40.5

У нас есть «grafana» и «prometheus», развернутые в кластере Kubernetes. Чтобы получить объекты Kubernetes, созданные после развертывания Grafana Helm Chart, выполните следующую команду:

```
$ kubectl get all
```

Команда дает следующий вывод:

NAME	READY	STATUS	RESTARTS	AGE
pod/grafana-65f7ddf46d-j4ch2	0/1	ContainerCreating	0	8m39s
pod/prometheus-alertmanager-0	1/1	Running	0	118m
pod/prometheus-kube-state-metrics-84bb75bf7d-xr51v	1/1	Running	2 (83m ago)	118m
pod/prometheus-prometheus-node-exporter-vn8b2	1/1	Running	1	118m
pod/prometheus-prometheus-pushgateway-7545cc8db-svz5f	1/1	Running	1	118m
pod/prometheus-server-7647d5bd9b-dct2n	2/2	Running	0	118m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/grafana	ClusterIP	10.111.230.98	<none>	80/TCP	9m13s
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	132m
service/prometheus-alertmanager	ClusterIP	10.104.50.77	<none>	9093/TCP	118m
service/prometheus-alertmanager-headless	ClusterIP	None	<none>	9093/TCP	118m
service/prometheus-kube-state-metrics	ClusterIP	10.109.139.238	<none>	8080/TCP	118m
service/prometheus-prometheus-node-exporter	ClusterIP	10.100.135.238	<none>	9100/TCP	118m
service/prometheus-prometheus-pushgateway	ClusterIP	10.103.224.84	<none>	9091/TCP	118m
service/prometheus-server	ClusterIP	10.104.207.27	<none>	80/TCP	118m
service/prometheus-server-ext	NodePort	10.101.163.153	<none>	80:31011/TCP	62m

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE
daemonset.apps/prometheus-prometheus-node-exporter	1	1	1	1	1	<none>	118m

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/grafana	0/1	1	0	1	9m5s	
deployment.apps/prometheus-kube-state-metrics	1/1	1	1	1	118m	
deployment.apps/prometheus-prometheus-pushgateway	1/1	1	1	1	118m	
deployment.apps/prometheus-server	1/1	1	1	1	118m	

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/grafana-65f7ddf46d	1	1	0	8m59s
replicaset.apps/prometheus-kube-state-metrics-84bb75bf7d	1	1	1	119m
replicaset.apps/prometheus-prometheus-pushgateway-7545cc8db	1	1	1	119m
replicaset.apps/prometheus-server-7647d5bd9b	1	1	1	119m

Мы создали модули, развертывания Kubernetes и сервисы для приложения Grafana Kubernetes.

Получите все созданные сервисы Kubernetes для приложения Grafana Kubernetes

Чтобы получить все созданные сервисы Kubernetes, выполните следующую команду:

```
$ kubectl get service
```

Он отображает следующие службы Kubernetes:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
grafana	ClusterIP	10.111.230.98	<none>	80/TCP	21m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	144m
prometheus-alertmanager	ClusterIP	10.104.50.77	<none>	9093/TCP	131m
prometheus-alertmanager-headless	ClusterIP	None	<none>	9093/TCP	131m
prometheus-kube-state-metrics	ClusterIP	10.109.139.238	<none>	8080/TCP	130m
prometheus-prometheus-node-exporter	ClusterIP	10.100.135.238	<none>	9100/TCP	130m
prometheus-prometheus-pushgateway	ClusterIP	10.103.224.84	<none>	9091/TCP	130m
prometheus-server	ClusterIP	10.104.207.27	<none>	80/TCP	130m
prometheus-server-ext	NodePort	10.101.163.153	<none>	80:31011/TCP	74m

Grafana Helm Chart создал сервис Kubernetes «grafana». Мы получим доступ к приложению Grafana с помощью службы Kubernetes «grafana».

Служба grafana имеет тип службы Kubernetes ClusterIP. Он позволяет получить доступ к приложению Grafana только внутри кластера Minikube Kubernetes.

Доступ к приложению Grafana Kubernetes

Чтобы получить доступ к приложению Grafana Kubernetes вне кластера Minikube Kubernetes, выполните следующую команду:

```
$ kubectl expose service grafana --type=NodePort --target-port=3000 --name=grafana-ext
```

Команда также преобразует службу Kubernetes Grafana-server из типа ClusterIP в тип NodePort. Это позволит нам получить доступ к приложению Grafana за пределами кластера Minikube через порт «3000».

Команда выводит следующее:

```
service/grafana-ext exposed
```

Чтобы проверить недавно созданную службу Kubernetes «grafana-ext», выполните следующую команду:

```
$ kubectl get service
```

Команда выводит следующее:

```

grafana          ClusterIP 10.111.230.98 <none> 80/TCP 26m
grafana-ext      NodePort 10.97.17.193 <none> 80:31130/TCP 42s
kubernetes       ClusterIP 10.96.0.1 <none> 443/TCP 149m
prometheus-alertmanager ClusterIP 10.104.50.77 <none> 9093/TCP 135m
prometheus-alertmanager-headless ClusterIP None <none> 9093/TCP 135m
prometheus-kube-state-metrics ClusterIP 10.109.139.238 <none> 8080/TCP 135m
prometheus-prometheus-node-exporter ClusterIP 10.100.135.238 <none> 9100/TCP 135m
prometheus-prometheus-pushgateway ClusterIP 10.103.224.84 <none> 9091/TCP 135m
prometheus-server ClusterIP 10.104.207.27 <none> 80/TCP 135m
prometheus-server-ext NodePort 10.101.163.153 <none> 80:31011/TCP 79m

```

Вы должны иметь доступ к сервису Kubernetes grafana-ext за пределами кластера Kubernetes, используя URL-адрес. Чтобы сгенерировать URL-адрес для доступа к этой службе, выполните следующую команду:

```
$ kubectl service grafana-ext
```

Команда выводит следующий URL:

```

! Executing "docker container inspect minikube --format={{.State.Status}}" took an unusually long time: 2.4407516s
! Restarting the docker service may improve performance.

```

NAMESPACE	NAME	TARGET PORT	URL
default	grafana-ext	80	http://192.168.49.2:31130

```

! Starting tunnel for service grafana-ext.

```

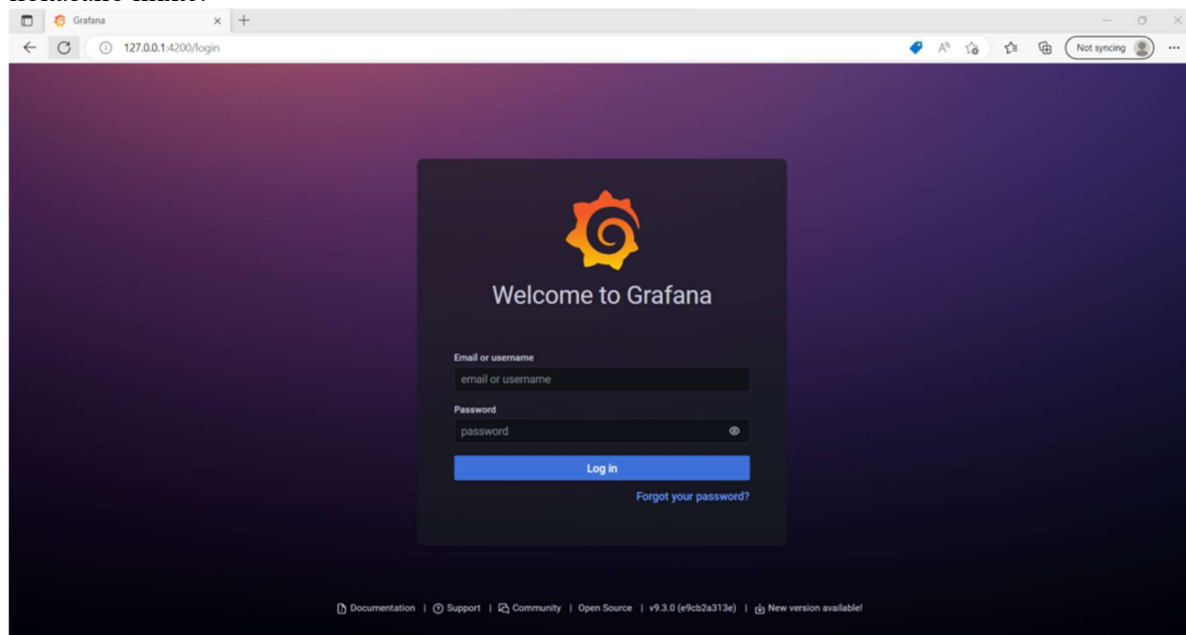
NAMESPACE	NAME	TARGET PORT	URL
default	grafana-ext		http://127.0.0.1:4200

```

! Opening service default/grafana-ext in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

```

Чтобы получить доступ к приложению Grafana, введите URL-адрес в браузере, как показано ниже:



На изображении выше мы установили приложение Grafana Helm Chart внутри кластера Kubernetes.

Модуль 19. Планирование безопасности для кластера Kubernetes

Задание: В k8s cluster из п.17 создать namespace vault и установить с помощью helm-chart hashicorp vault последней версии, настроить под него persistent volume с 10 GB.

Уровень: 10

Продолжительность выполнения: 40 мин.

Пространства имен Kubernetes

Мы рекомендуем использовать пространство имен, отличное от пространства имен по умолчанию, для развертывания приложений в рабочей среде по таким причинам, как изоляция, управление доступом, управление обновлением и общее логическое разделение элементов Kubernetes по умолчанию, а также общая помощь в долгосрочном управлении.

Пример:

Создайте пространство имен K8s.

```
$ kubectl create namespace vault
```

Просмотрите свои новые объекты K8s.

```
$ kubectl --namespace='vault' get all
```

Настройка репозитория Helm

Чтобы получить доступ к диаграмме Vault Helm, добавьте репозиторий Hashicorp Helm.

```
$ helm repo add hashicorp https://helm.releases.hashicorp.com
"hashicorp" has been added to your repositories
```

Убедитесь, что у вас есть доступ к графику.

```
$ helm search repo hashicorp/vault
NAME                CHART VERSION APP VERSION DESCRIPTION
hashicorp/vault     0.6.0          1.4.3          Install and configure Vault
on Kubernetes.
```

Использование диаграмм Helm

Команде `helm install` требуются параметры `[release name]`, `[repo/chart]`, и есть возможность `--namespace` объявить пространство имен Kubernetes для запуска команды. Обычно диаграммы Helm находятся в стадии значительной разработки, поэтому мы рекомендуем сначала запускать Helm `--dry-run` перед любой установкой или обновлением, чтобы проверить изменения, если вы не указываете более старую версию. Этот `--dry-run` флаг заставит Helm распечатать результирующие манифесты YAML, которые логически создает и применяет диаграмма Helm. Поскольку потребности одной компании не совпадают с потребностями другой, `helm install` команда также принимает параметры для переопределения значений конфигурации по умолчанию, встроенных или определенных в файле YAML.

Примеры:

```
$ helm install vault hashicorp/vault --namespace vault --dry-run
```

Список доступных выпусков.

```
$ helm search repo hashicorp/vault --versions
NAME          CHART VERSION APP VERSION DESCRIPTION
hashicorp/vault 0.6.0          1.4.3      Install and configure Vault
on Kubernetes.
hashicorp/vault 0.5.0          1.4.3      Install and configure Vault
on Kubernetes.
```

Установите версию 0.5.0.

```
$ helm install vault hashicorp/vault --namespace vault --version 0.5.0
```

Переопределить настройки по умолчанию.

```
$ helm install vault hashicorp/vault \
  --namespace vault \
  --set "server.ha.enabled=true" \
  --set "server.ha.replicas=5" \
  --dry-run
```

Или укажите желаемую конфигурацию в файле `override-values.yml`.

```
$ cat << EOF > ./override-values.yml
server:
  ha:
    enabled: true
    replicas: 5
EOF
```

Переопределите конфигурацию по умолчанию значениями, считанными из `override-values.yml` файла.

```
$ helm install vault hashicorp/vault \
  --namespace vault \
  -f override-values.yml \
  --dry-run
```

Настройка диаграммы Vault Helm

Для установки, готовой к работе, мы предлагаем установить диаграмму Vault Helm в режиме [высокой доступности](#) (HA). При этом устанавливается StatefulSet серверных модулей Vault либо с интегрированным хранилищем, либо с серверной частью хранилища Consul. Однако диаграмма Vault Helm также может настроить Vault для работы в [режиме разработки](#) или в [автономном](#) режиме.

В этом руководстве мы продемонстрируем установку в режиме высокой готовности с интегрированным хранилищем.

Примечание

Vault Integrated Storage реализует протокол хранения Raft и обычно упоминается как **Raft** в документации HashiCorp Vault. При использовании режима HA с серверной частью хранилища Consul мы также рекомендуем использовать [диаграмму Consul Helm](#). В приведенном ниже `override-values.yml` файле представлен набор значений для атрибутов, которые обычно переопределяются при развертывании Vault в рабочей среде Kubernetes. Создайте приведенный ниже `override-values.yml` файл, и мы обсудим

его и, наконец, установим последнюю диаграмму Vault Helm в режиме HA Integrated Storage, используя эти переопределения.

```
$ cat << EOF > ./override-values.yml

# Vault Helm Chart Value Overrides
global:
  enabled: true
  tlsDisable: false

injector:
  enabled: true
  # Use the Vault K8s Image https://github.com/hashicorp/vault-k8s/
  image:
    repository: "hashicorp/vault-k8s"
    tag: "latest"

  resources:
    requests:
      memory: 256Mi
      cpu: 250m
    limits:
      memory: 256Mi
      cpu: 250m

server:
  # Use the Enterprise Image
  image:
    repository: "hashicorp/vault-enterprise"
    tag: "1.5.0_ent"

  # These Resource Limits are in line with node requirements in the
  # Vault Reference Architecture for a Small Cluster
  resources:
    requests:
      memory: 8Gi
      cpu: 2000m
    limits:
      memory: 16Gi
      cpu: 2000m

  # For HA configuration and because we need to manually init the
  vault,
  # we need to define custom readiness/liveness Probe settings
  readinessProbe:
    enabled: true
    path:
"/v1/sys/health?standbyok=true&sealedcode=204&uninitcode=204"
  livenessProbe:
    enabled: true
    path: "/v1/sys/health?standbyok=true"
    initialDelaySeconds: 60

  # extraEnvironmentVars is a list of extra environment variables to
  set with the stateful set. These could be
  # used to include variables required for auto-unseal.
  extraEnvironmentVars:
```

```

    VAULT_CACERT: /vault/userconfig/tls-ca/ca.crt

    # extraVolumes is a list of extra volumes to mount. These will be
    exposed
    # to Vault in the path `/vault/userconfig/<name>/`.
    extraVolumes:
      - type: secret
        name: tls-server
      - type: secret
        name: tls-ca
      - type: secret
        name: kms-creds

    # This configures the Vault Statefulset to create a PVC for audit
    logs.
    # See https://www.vaultproject.io/docs/audit/index.html to know more
    auditStorage:
      enabled: true

    standalone:
      enabled: false

    # Run Vault in "HA" mode.
    ha:
      enabled: true
      replicas: 5
      raft:
        enabled: true
        setNodeId: true

    config: |
      ui = true
      listener "tcp" {
        address = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_cert_file = "/vault/userconfig/tls-server/fullchain.pem"
        tls_key_file = "/vault/userconfig/tls-server/server.key"
        tls_client_ca_file = "/vault/userconfig/tls-server/client-
auth-ca.pem"
      }

      storage "raft" {
        path = "/vault/data"
        retry_join {
          leader_api_addr = "https://vault-0.vault-internal:8200"
          leader_ca_cert_file = "/vault/userconfig/tls-ca/ca.crt"
          leader_client_cert_file = "/vault/userconfig/tls-
server/server.crt"
          leader_client_key_file = "/vault/userconfig/tls-
server/server.key"
        }
        retry_join {
          leader_api_addr = "https://vault-1.vault-internal:8200"
          leader_ca_cert_file = "/vault/userconfig/tls-ca/ca.crt"
          leader_client_cert_file = "/vault/userconfig/tls-
server/server.crt"
          leader_client_key_file = "/vault/userconfig/tls-
server/server.key"

```

```

    }
    retry_join {
        leader_api_addr = "https://vault-2.vault-internal:8200"
        leader_ca_cert_file = "/vault/userconfig/tls-ca/ca.crt"
        leader_client_cert_file = "/vault/userconfig/tls-
server/server.crt"
        leader_client_key_file = "/vault/userconfig/tls-
server/server.key"
    }
    retry_join {
        leader_api_addr = "https://vault-3.vault-internal:8200"
        leader_ca_cert_file = "/vault/userconfig/tls-ca/ca.crt"
        leader_client_cert_file = "/vault/userconfig/tls-
server/server.crt"
        leader_client_key_file = "/vault/userconfig/tls-
server/server.key"
    }
    retry_join {
        leader_api_addr = "https://vault-4.vault-internal:8200"
        leader_ca_cert_file = "/vault/userconfig/tls-ca/ca.crt"
        leader_client_cert_file = "/vault/userconfig/tls-
server/server.crt"
        leader_client_key_file = "/vault/userconfig/tls-
server/server.key"
    }

    autopilot {
        cleanup_dead_servers = "true"
        last_contact_threshold = "200ms"
        last_contact_failure_threshold = "10m"
        max_trailing_logs = 250000
        min_quorum = 5
        server_stabilization_time = "10s"
    }

}

service_registration "kubernetes" {}

# Vault UI
ui:
    enabled: true
    serviceType: "LoadBalancer"
    serviceNodePort: null
    externalPort: 8200

    # For Added Security, edit the below
    #loadBalancerSourceRanges:
    #   - < Your IP RANGE Ex. 10.0.0.0/16 >
    #   - < YOUR SINGLE IP Ex. 1.78.23.3/32 >

EOF

```